TBU

Zhuo Diao, Nikita Gupta

March 30, 2026

1 Set Up

```
> # Install remotes if you don't have it
> if (!require("remotes")) install.packages("remotes")
>
> # Install the micd package from the BIPS repository
> remotes::install_github("bips-hb/micd")
>
> #  Load the library
> library(micd)
> library(pcalg)
```

```
> library(tidyverse) #installed tidyverse
> library(nlme)
> library(ggplot2)
> library(here)
> library(kableExtra)
> library(modelsummary)
> library(dplyr)
> library(performance)
> library(glmnet)
> library(pcalg)
> library(gridExtra)
> library(bnlearn)
> library(igraph)
> library(gt)
> library(gtExtras)
> library(corrplot)
> library(ggpubr)
> library(dagitty)
>
> bootstrap_sample_size <- 1000
>
> set.seed(516)
> getwd()
```

```
[1] "/Users/sunilmd/Desktop/vscode/MATH-516/project-3-nikita-zhuo/ANALYSIS_TESTING_1ALPHA_2OPTA"
```

```
> data <- read.csv(here("low2076upd.csv"))
> colSums(is.na(data)) #no NAs in any columns
```

```
       Y         A LIMIT_BAL       SEX EDUCATION       AGE     PAY_0     PAY_2
       0         0         0         0         0         0         0         0
```

```
    PAY_3     PAY_4     PAY_5     PAY_6 BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4
        0         0         0         0         0         0         0         0
BILL_AMT5 BILL_AMT6  PAY_AMT1  PAY_AMT2  PAY_AMT3  PAY_AMT4  PAY_AMT5  PAY_AMT6
        0         0         0         0         0         0         0         0
```

```
> nrow(data) #500 rows
```

```
[1] 500
```

```
> colnames(data)
```

```
 [1] "Y"         "A"         "LIMIT_BAL" "SEX"       "EDUCATION" "AGE"
 [7] "PAY_0"     "PAY_2"     "PAY_3"     "PAY_4"     "PAY_5"     "PAY_6"
[13] "BILL_AMT1" "BILL_AMT2" "BILL_AMT3" "BILL_AMT4" "BILL_AMT5" "BILL_AMT6"
[19] "PAY_AMT1"  "PAY_AMT2"  "PAY_AMT3"  "PAY_AMT4"  "PAY_AMT5"  "PAY_AMT6"
```

2 Data manipulation

```
> #factorize cateogorical variables
> data$Y <- as.factor(data$Y)
> data$A <- as.factor(data$A)
> data$SEX <- as.factor(data$SEX)
> data$EDUCATION <- as.factor(data$EDUCATION)
> data$PAY_0 <- as.factor(data$PAY_0)
> data$PAY_2 <- as.factor(data$PAY_2)
> data$PAY_3 <- as.factor(data$PAY_3)
> data$PAY_4 <- as.factor(data$PAY_4)
> data$PAY_5 <- as.factor(data$PAY_5)
> data$PAY_6 <- as.factor(data$PAY_6)
```

```
> #create list of ALL covariates - everything except A and Y
> cov_all <- setdiff(names(data), c("Y", "A"))
> print(cov_all)
```

```
 [1] "LIMIT_BAL" "SEX"       "EDUCATION" "AGE"       "PAY_0"     "PAY_2"
 [7] "PAY_3"     "PAY_4"     "PAY_5"     "PAY_6"     "BILL_AMT1" "BILL_AMT2"
[13] "BILL_AMT3" "BILL_AMT4" "BILL_AMT5" "BILL_AMT6" "PAY_AMT1"  "PAY_AMT2"
[19] "PAY_AMT3"  "PAY_AMT4"  "PAY_AMT5"  "PAY_AMT6"
```

```
> # Create a version of the data where everyone is enrolled, and no one is enrolled, for outcome regres
> data_all_enrolled <- data
> data_all_enrolled$A <- 1
> data_all_enrolled$A <- as.factor(data_all_enrolled$A)
>
> data_no_enrolled <- data
> data_no_enrolled$A <- 0
> data_no_enrolled$A <- as.factor(data_no_enrolled$A)
> print(unique(data_all_enrolled$A)) #looks good
```

```
[1] 1
Levels: 1
```

```
> print(unique(data_no_enrolled$A))
```

```
[1] 0
Levels: 0
```

3 EDA

4 1 Now the Analysis, for all variables!

5 All Variables: First, untransformed data

6 All Variables, Outcome regression

Basically, model the outcome (Y) of defaulting.

1. Train the model to understand how every variable including PPP enrollment A affects prob of defaulting Y.
2. Take all my customers (A=0 and A=1), and see what their default rate would be if they had A=1.
3. Take all my customers (A=0 and A=1), and see what their default rate would be if they had A=0.
4. ACE is the difference between the two.

```
> formula_or_all_var <- as.formula(paste("Y ~ A +", paste(cov_all, collapse = "+"))) #now A is part of
> model_or_all_var <- glm(formula_or_all_var, data = data, family = binomial) #binomial since 0 or 1 de
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_or_all_var)
```

```
Y ~ A + LIMIT_BAL + SEX + EDUCATION + AGE + PAY_0 + PAY_2 + PAY_3 +
    PAY_4 + PAY_5 + PAY_6 + BILL_AMT1 + BILL_AMT2 + BILL_AMT3 +
    BILL_AMT4 + BILL_AMT5 + BILL_AMT6 + PAY_AMT1 + PAY_AMT2 +
    PAY_AMT3 + PAY_AMT4 + PAY_AMT5 + PAY_AMT6
```

```
> # Predict default probability for world of everyone enrolled, and no one enrolled
> data$prob_Y_if_A1_all_var <- predict(model_or_all_var, newdata = data_all_enrolled, type = "response")
> EY_a1_or_all_var <- mean(data$prob_Y_if_A1_all_var)
> data$prob_Y_if_A0_all_var <- predict(model_or_all_var, newdata = data_no_enrolled, type = "response")
> EY_a0_or_all_var <- mean(data$prob_Y_if_A0_all_var)
>
> ace_or_all_var <- EY_a1_or_all_var - EY_a0_or_all_var
>
> #print means
> print("Default likelihood (Full Set, OR), A=1 then A=0")
```

```
[1] "Default likelihood (Full Set, OR), A=1 then A=0"
```

```
> print(EY_a1_or_all_var)
```

```
[1] 0.424858
```

```
> print(EY_a0_or_all_var)
```

```
[1] 0.2184891
```

```
> print(paste("OR ACE (Full Set):", round(ace_or_all_var,4)))
```

```
[1] "OR ACE (Full Set): 0.2064"
```

OR ACE for all vars is 0.2108, meaning enrolling in the PPP is associated with 21% increase in probability of missing a payment. This seems unintuitive…

7 All Variables, IPW

Basically, we know that PPP enrollment is not split evenly between low and high risk customers. We want a way to equalize for underlying riskiness. So we calculate a propensity score to see the likelihood of a customer being enrolled in the PPP. We then look at what ACTUALLY happened (if they were enrolled, and if they charged off). We divide that by the propensity score to ensure that low population outcomes / segments are properly represented/weighted.

1. (= L) = (L) = (L;): Model the probability a customer is enrolled in the PPP, given their covariates. Basically, use a logistic regression where response variable is A (PPP enrollment), and all variables except A and Y are predictors. Results in function that assigns every customer a probability of being enrolled in PPP - propensity score.
2. I(A=a): indicator flagging to only look at people who actually received the treatment we're interested in. So, if A=1, only look at people who were actually enrolled.
3. Y: actual missed payment status, for everyone the indicator flags. Overall: Someone had a 10% chance of enrolling but actually did enroll (A=1), and did NOT miss their pmt (Y=0): denominator is 0.1, while numerator is 1. So now their missed payment status is 10x the average person in terms of weight.

```
> #get propensity scores
> formula_ps_all_var <- as.formula(paste("A~", paste(cov_all, collapse = "+"))) #formula of A~all covar
> model_ps_all_var <- glm(formula_ps_all_var, data = data, family = binomial) #binomial because only 2
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_ps_all_var)
```

```
A ~ LIMIT_BAL + SEX + EDUCATION + AGE + PAY_0 + PAY_2 + PAY_3 +
    PAY_4 + PAY_5 + PAY_6 + BILL_AMT1 + BILL_AMT2 + BILL_AMT3 +
    BILL_AMT4 + BILL_AMT5 + BILL_AMT6 + PAY_AMT1 + PAY_AMT2 +
    PAY_AMT3 + PAY_AMT4 + PAY_AMT5 + PAY_AMT6
```

```
> data$ps_all_var <- predict(model_ps_all_var, type = "response") #calculate propensity scores for all
> print("min and max ps")
```

```
[1] "min and max ps"
```

```
> print(min(data$ps_all_var)) #.199515e-08
```

```
[1] 1.127689e-08
```

```
> print(max(data$ps_all_var)) #1
```

```
[1] 1
```

```
> #extreme ps means likely have extreme scores
>
> data$ps_all_var_clamped <- pmax(pmin(data$ps_all_var, 0.9999), 0.0001) #clamped ps; if 0 or 1 then mo
>
>
> # calculate weights.
> data$weights_all_var <- ifelse(data[["A"]] == 1, 1 / data$ps_all_var_clamped, 1 / (1 - data$ps_all_va
> print("min and max weights")
```

```
[1] "min and max weights"
```

```
> print(min(data$weights_all_var))
```

```
[1] 1.0001
```

```
> print(max(data$weights_all_var))
```

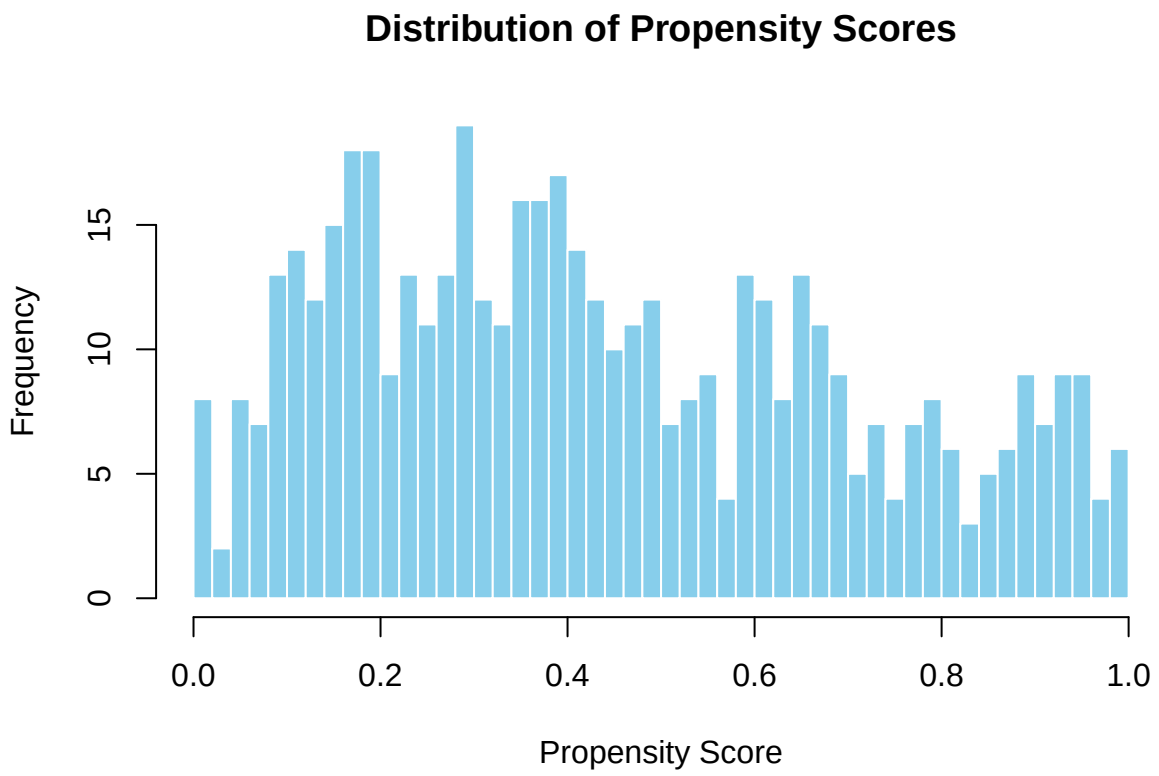
```
[1] 25.74811
```

```
> #if customer was actually enrolled (A==1), then d weight as 1/ps, otherwise do 1/(1-ps)
> #So if customer had A=1 & ps=0.8, then weight = 1.25; but if A=0 then weight=5. Scales by unexpectedn
>
> # Estimate ACE using the weighted average
> #filter to only people who were enrolled A=1 or A=0
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
> # Filter to A0 or A1 groups, get default and multiply by weight; basically, if paid on time, contribu
> EY_a0_ipw_all_var <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_all_var)/sum(a0_grou
> EY_a1_ipw_all_var <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_all_var)/sum(a1_grou
>
> ace_ipw_all_var <- EY_a1_ipw_all_var - EY_a0_ipw_all_var
> print(paste("IPW ACE (Full Set):", round(ace_ipw_all_var,4)))
```

```
[1] "IPW ACE (Full Set): 0.2256"
```

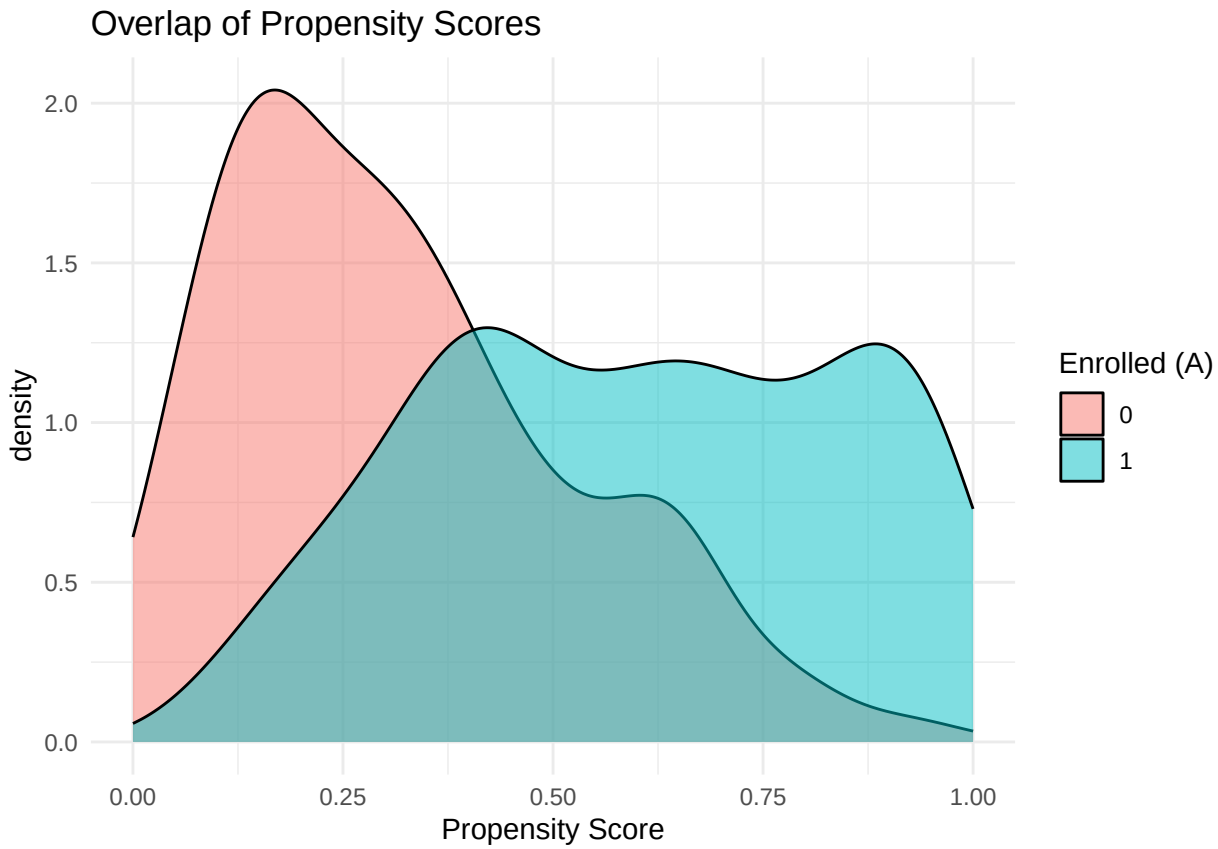
Soem graphs are below just to look at distribution of scores, etc. Given those graphs and the model error from bootstrapping, I had decided to clamp the propensity scores because otherwise we would potentially get a divide by 0 error.

```
> # Create a histogram of Propensity Scores
> hist(data$ps_all_var,
+      breaks = 50,
+      main = "Distribution of Propensity Scores",
+      xlab = "Propensity Score",
+      col = "skyblue",
+      border = "white")
```

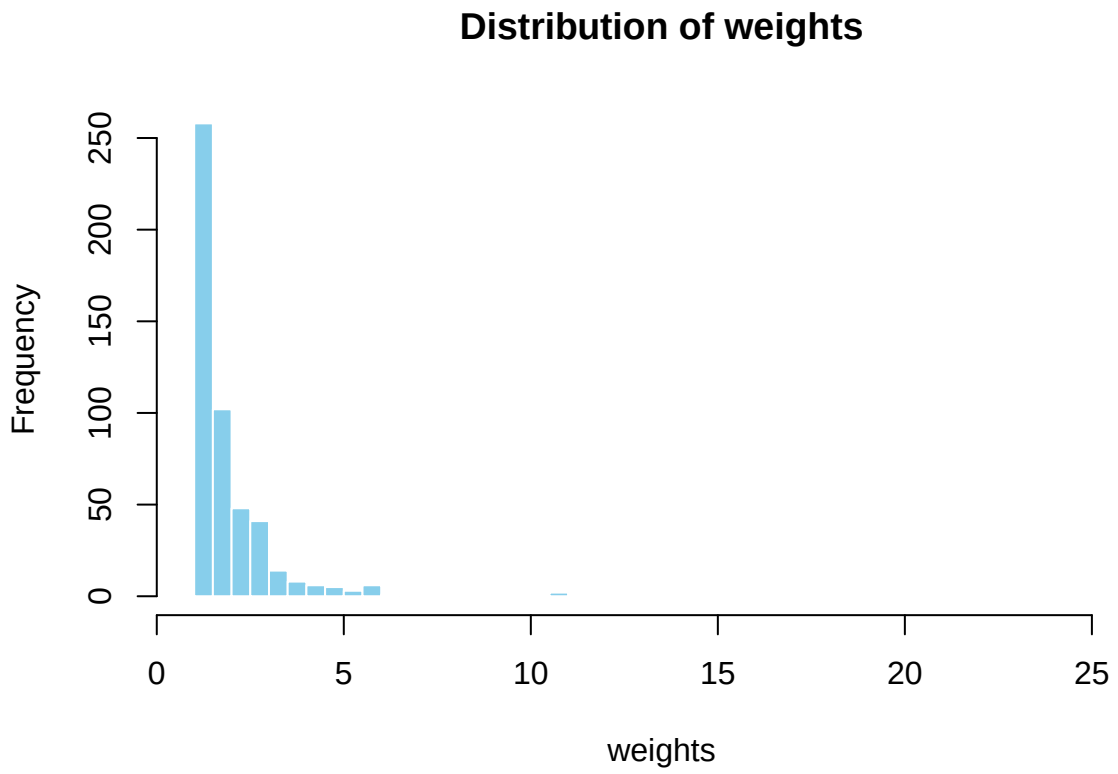



```
> #a lot of low propensity scores
```

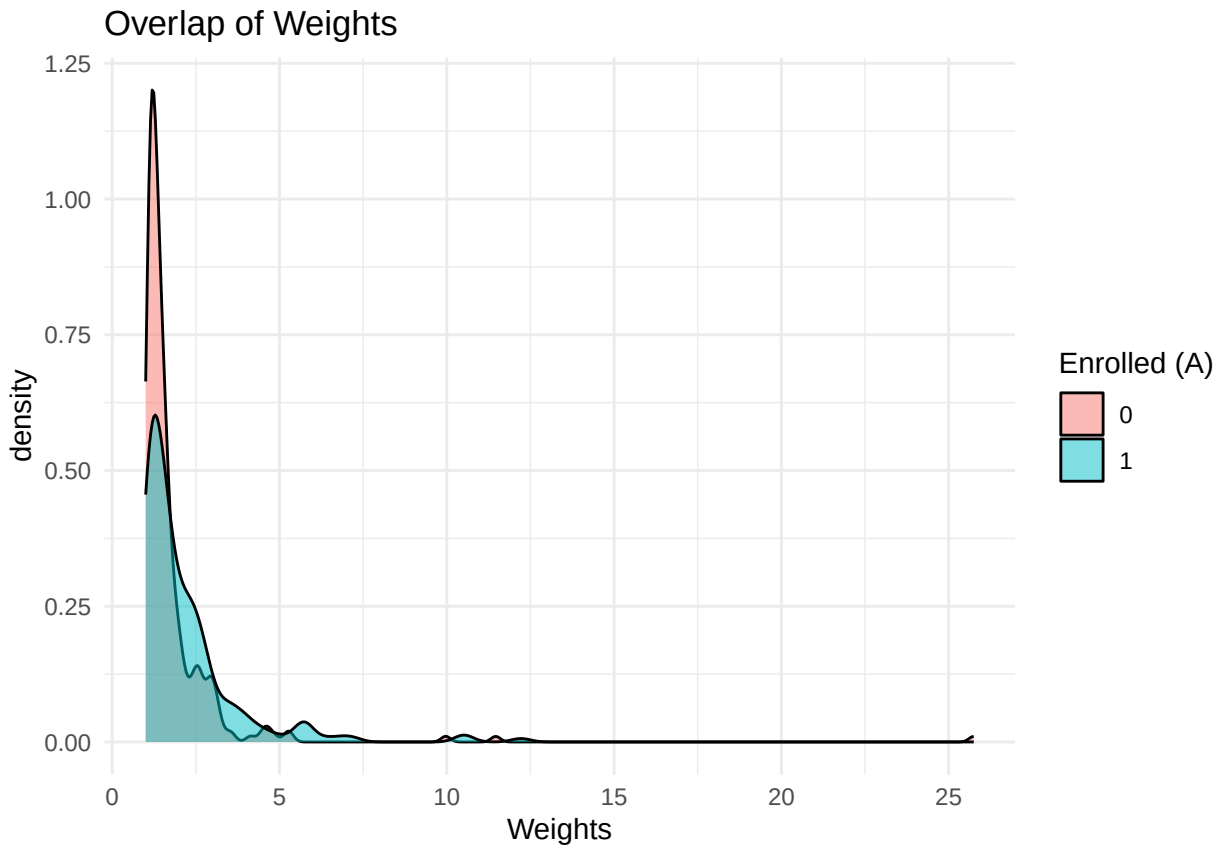
```
> #Plot by Treatment Group to see "Overlap"
> ggplot(data, aes(x = ps_all_var, fill = as.factor(A))) +
+   geom_density(alpha = 0.5) +
+   labs(title = "Overlap of Propensity Scores",
+        x = "Propensity Score",
+        fill = "Enrolled (A)") +
+   theme_minimal()
```




```
> #plot weights
> hist(data$weights_all_var,
+      breaks = 50,
+      main = "Distribution of weights",
+      xlab = "weights",
+      col = "skyblue",
+      border = "white")
```




```
> ggplot(data, aes(x = weights_all_var, fill = as.factor(A))) +
+   geom_density(alpha = 0.5) +
+   labs(title = "Overlap of Weights",
+        x = "Weights",
+        fill = "Enrolled (A)") +
+   theme_minimal()
```




```
> #no real takeaway. A=1 has higher weights in general maybe?
```

```
> #see how weights are distr A=1 vs A=0
> data %>%
+   group_by(A) %>%
+   summarise(
+     Count = n(),
+     Min     = min(weights_all_var, na.rm = TRUE),
+     Average = mean(weights_all_var, na.rm = TRUE),
+     Median  = median(weights_all_var, na.rm = TRUE),
+     Perc_25 = quantile(weights_all_var, 0.25, na.rm = TRUE),
+     Perc_75 = quantile(weights_all_var, 0.75, na.rm = TRUE),
+     Max     = max(weights_all_var, na.rm = TRUE),
+     na_count = sum(is.na(weights_all_var)),
+     inf_count = sum(is.infinite(weights_all_var))
+   )
```

```
# A tibble: 2 x 10
  A     Count   Min Average Median Perc_25 Perc_75   Max na_count inf_count
  <fct> <int> <dbl>   <dbl>  <dbl>   <dbl>   <dbl> <dbl>    <int>     <int>
1 0       279  1.00    1.79   1.39    1.18    1.80  25.7        0         0
2 1       221  1.00    2.18   1.66    1.22    2.46  12.2        0         0
```

```
> #although max weight for A=0 is bigger, overall A=1 tends to have higher weights given perc_75, media
> #no NAs or inf
```

8 All Variables, bootstrap,

Bootstrap to get the bootstrap bias (unsure if important) and variance!

Going forward, I will do bootstrapping so I can capture variance, etc.

Note that I clamp PS scores below, and throughout the rest.

```
> bootstrap_results_all_var <- replicate(bootstrap_sample_size, {
+   #create bootstrap sample
+   sample_df <- data[sample(nrow(data), replace = TRUE), ]
+
+   #outcome regression
+   #run model
+   formula_or_sample_df <- as.formula(paste("Y ~ A +", paste(cov_all, collapse = "+")))
+   model_or_sample_df <- glm(formula_or_sample_df, data = sample_df, family = binomial)
+
+   #create fake datasets
+   sample_df_a0 <- sample_df
+   sample_df_a0$A <- factor(0, levels = levels(sample_df$A))
+   sample_df_a1 <- sample_df
+   sample_df_a1$A <- factor(1, levels = levels(sample_df$A))
+
+   # Predict default probability for world of everyone enrolled, and no one enrolled
+   sample_df_prob_Y_if_A1 <- predict(model_or_sample_df, newdata = sample_df_a1, type = "response")
+   EY_a1_or_sample_df <- mean(sample_df_prob_Y_if_A1)
+
+   sample_df_prob_Y_if_A0 <- predict(model_or_sample_df, newdata = sample_df_a0, type = "response")
+   EY_a0_or_sample_df <- mean(sample_df_prob_Y_if_A0)
+
+   ace_or_sample_df <- EY_a1_or_sample_df - EY_a0_or_sample_df
+
+   #IPW
+   #get propensity score
+   formula_ps_sample_df <- as.formula(paste("A~", paste(cov_all, collapse = "+")))
+   model_ps_sample_df <- glm(formula_ps_sample_df, data = sample_df, family = binomial)
+   sample_df$ps_sample_df <- predict(model_ps_sample_df, type = "response")
+   #clamp ps score between 0.0001 and 0.9999 because otherwise get warnings of fitted prob being 0 or
+   sample_df$ps_sample_df_clamped <- pmax(pmin(sample_df$ps_sample_df, 0.9999), 0.0001)
+   #get weights
+   sample_df$weights_sample_df <- ifelse(sample_df$A == 1, 1/sample_df$ps_sample_df_clamped, 1/(1-samp
+   #divide into a0 and a1 groups
+   a0_group_sample_df <- sample_df[sample_df$A == 0, ]
+   a1_group_sample_df <- sample_df[sample_df$A == 1, ]
+   #calculate ACE
+   EY_a0_ipw_sample_df <- sum(as.numeric(as.character(a0_group_sample_df$Y)) * a0_group_sample_df$weig
+   EY_a1_ipw_sample_df <- sum(as.numeric(as.character(a1_group_sample_df$Y)) * a1_group_sample_df$weig
+   ace_ipw_sample_df <-  EY_a1_ipw_sample_df - EY_a0_ipw_sample_df
+   return(c(ace_or_sample_df, ace_ipw_sample_df))
+ })
```

```
> # print(bootstrap_results_all_var)
```

Basically, this outputs 2 rows of data ([1,], [2,]). With the 1st row being OR and 2nd being IPW method. and then 10 columns (or however many runs of columns).

```
> orig_ace_all_var <- c(ace_or_all_var, ace_ipw_all_var)
>
> # convert to dataframe
> boot_df_all_var <- as.data.frame(t(bootstrap_results_all_var))
> colnames(boot_df_all_var) <- c("Regression", "IPW")
>
> # Calculate Metrics
> metrics_all_var <- data.frame(
+   Method = c("Regression", "IPW"),
+   Original_ACE = orig_ace_all_var, # Use values from the full dataset
+   Bootstrap_ACE = c(mean(boot_df_all_var$Regression), mean(boot_df_all_var$IPW)),
+   Standard_Error = c(sd(boot_df_all_var$Regression), sd(boot_df_all_var$IPW))
+ )
>
> # Calculate Bias
> metrics_all_var$Bootstrap_Bias <- metrics_all_var$Bootstrap_ACE - metrics_all_var$Original_ACE
> print(metrics_all_var)
```

```
      Method Original_ACE Bootstrap_ACE Standard_Error Bootstrap_Bias
1 Regression    0.2063689     0.2146816     0.04388369     0.00831272
2        IPW    0.2255985     0.2362782     0.06317165     0.01067971
```

This suggests that regression results in a much more stable result, although it is very unintuitive. IPW results in a much higher variance; even with this variance, it is a more intuitive answer as it still suggests a payment plan decreases default risk. Regression is probably biased by overfitting noise and thus creating bias (no proof here, just judgemental unintuitiveness), while IPW could be distorted by extreme weights (as evidenced by high standard error).

9 2 Now the Analysis, for lasso-selected variables!

10 Lasso Variables, Outcome regression , not transformed

```
> #set up the data
> x_or_lasso <- model.matrix(Y ~ A + ., data = data[, c("Y", "A", cov_all)])[,-1] #only pull in cov_all
> y_or_lasso <- data$Y
> print(colnames(x_or_lasso))
```

```
 [1] "A1"         "LIMIT_BAL"  "SEX1"       "EDUCATION2" "EDUCATION3"
 [6] "EDUCATION4" "EDUCATION5" "AGE"        "PAY_0-1"    "PAY_00"
[11] "PAY_01"     "PAY_02"     "PAY_03"     "PAY_04"     "PAY_06"
[16] "PAY_2-1"    "PAY_20"     "PAY_22"     "PAY_23"     "PAY_24"
[21] "PAY_25"     "PAY_3-1"    "PAY_30"     "PAY_32"     "PAY_33"
[26] "PAY_34"     "PAY_4-1"    "PAY_40"     "PAY_42"     "PAY_43"
[31] "PAY_5-1"    "PAY_50"     "PAY_52"     "PAY_53"     "PAY_54"
[36] "PAY_6-1"    "PAY_60"     "PAY_62"     "PAY_63"     "BILL_AMT1"
[41] "BILL_AMT2"  "BILL_AMT3"  "BILL_AMT4"  "BILL_AMT5"  "BILL_AMT6"
[46] "PAY_AMT1"   "PAY_AMT2"   "PAY_AMT3"   "PAY_AMT4"   "PAY_AMT5"
[51] "PAY_AMT6"
```

```
> col_idx_A1 <- which(colnames(x_or_lasso) == "A1")
> p_fac_or <- rep(1, ncol(x_or_lasso))
> p_fac_or[col_idx_A1] <- 0
>
```

```
> #for A in (penalty factor 0)
> #fit lasso with cross-validation, get best lambda
> model_or_lasso <- cv.glmnet(x_or_lasso, y_or_lasso, family = "binomial", alpha = 1, penalty.factor = p
> best_lambda_or_lasso <- model_or_lasso$lambda.min
> print(coef(model_or_lasso))

52 x 1 sparse Matrix of class "dgCMatrix"
             lambda.1se
(Intercept) -1.36384461
A1           0.91904854
LIMIT_BAL    .
SEX1         .
EDUCATION2   .
EDUCATION3   .
EDUCATION4   .
EDUCATION5   .
AGE          .
PAY_0-1      .
PAY_00       .
PAY_01       .
PAY_02       0.30689616
PAY_03       .
PAY_04       .
PAY_06       .
PAY_2-1      .
PAY_20       .
PAY_22       .
PAY_23       .
PAY_24       .
PAY_25       .
PAY_3-1      .
PAY_30       .
PAY_32       0.78596628
PAY_33       .
PAY_34       .
PAY_4-1      .
PAY_40       .
PAY_42       .
PAY_43       .
PAY_5-1      .
PAY_50       .
PAY_52       .
PAY_53       .
PAY_54       .
PAY_6-1      .
PAY_60       .
PAY_62       0.02140787
PAY_63       .
BILL_AMT1    .
BILL_AMT2    .
BILL_AMT3    .
BILL_AMT4    .
BILL_AMT5    .
BILL_AMT6    .
```

```
PAY_AMT1     .
PAY_AMT2     .
PAY_AMT3     .
PAY_AMT4     .
PAY_AMT5     .
PAY_AMT6     .
```

```
> # Predict default probability for world of everyone enrolled, and no one enrolled
>
> #basically, A1 is a dummy column for the factor A=1
> #get the column number for that
> # create new datasets where A is overridden to be all 1s or all 0s, and then transform into model mat
> x_all_enrolled_or_lasso <- x_or_lasso
> x_all_enrolled_or_lasso[, col_idx_A1] <- 1  # Everyone treated
> x_no_enrolled_or_lasso <- x_or_lasso
> x_no_enrolled_or_lasso[, col_idx_A1] <- 0  # Everyone untreated
>
> #now predict
> data$prob_Y_if_A1_lasso <- as.numeric(predict(model_or_lasso, s = best_lambda_or_lasso, newx = x_all_
> EY_a1_or_lasso <- mean(data$prob_Y_if_A1_lasso)
>
> data$prob_Y_if_A0_lasso <- as.numeric(predict(model_or_lasso, s = best_lambda_or_lasso, newx = x_no_e
> EY_a0_or_lasso <- mean(data$prob_Y_if_A0_lasso)
>
>
> ace_or_lasso <- EY_a1_or_lasso - EY_a0_or_lasso
>
> print("formula")
```

```
[1] "formula"
```

```
> lasso_y_coefs <- coef(model_or_lasso, s = "lambda.min")
> lasso_y_vars <- rownames(lasso_y_coefs)[which(as.matrix(lasso_y_coefs) != 0)]
> # Remove Intercept and Treatment A to get the adjustment set
> cov_lasso_y <- setdiff(lasso_y_vars, c("(Intercept)", "A"))
> formula_lasso_or_text <- paste("Y ~ ", paste(cov_lasso_y, collapse = " + "))
> formula_lasso_or_text_anotherver <- lasso_y_vars[which(lasso_y_vars != "(Intercept)")]
> print(formula_lasso_or_text)
```

```
[1] "Y ~  A1 + LIMIT_BAL + EDUCATION2 + EDUCATION4 + AGE + PAY_0-1 + PAY_00 + PAY_01 + PAY_02 + PAY_03 +
```

```
> print(lasso_y_vars)
```

```
 [1] "(Intercept)" "A1"          "LIMIT_BAL"   "EDUCATION2"  "EDUCATION4"
 [6] "AGE"         "PAY_0-1"     "PAY_00"      "PAY_01"      "PAY_02"
[11] "PAY_03"      "PAY_06"      "PAY_24"      "PAY_25"      "PAY_32"
[16] "PAY_34"      "PAY_4-1"     "PAY_42"      "PAY_43"      "PAY_54"
[21] "PAY_62"      "BILL_AMT5"   "PAY_AMT2"    "PAY_AMT3"    "PAY_AMT4"
[26] "PAY_AMT6"
```

```
> #print means
> print("Default likelihood (Lasso, OR), A=1 then A=0")
```

```
[1] "Default likelihood (Lasso, OR), A=1 then A=0"
```

```
> print(EY_a1_or_lasso)
```

```
[1] 0.4243885
```

```
> print(EY_a0_or_lasso)
```

```
[1] 0.2211227
```

```
> print(paste("OR ACE (Lasso):", round(ace_or_lasso,4)))
```

```
[1] "OR ACE (Lasso): 0.2033"
```

11 Lasso Variables, IPW

```
> #prepare data
> x_ps_lasso <- model.matrix(A ~ ., data = data[, c("A", cov_all)])[,-1]
> y_ps_lasso <- data$A
>
>
> #fit lasso
> model_ps_lasso <- cv.glmnet(x_ps_lasso, y_ps_lasso, family = "binomial", alpha = 1)
> best_lambda_ps_lasso <- model_ps_lasso$lambda.min
>
>
> #get propensity scores
> data$ps_lasso <- as.numeric(predict(model_ps_lasso, s = best_lambda_ps_lasso, newx = x_ps_lasso, type
>
>
> print("min and max ps")
```

```
[1] "min and max ps"
```

```
> print(min(data$ps_lasso))
```

```
[1] 0.1381571
```

```
> print(max(data$ps_lasso))
```

```
[1] 0.9413102
```

```
> data$ps_lasso_clamped <- pmax(pmin(data$ps_lasso, 0.9999), 0.0001)
>
>
> # calculate weights.
> data$weights_lasso <- ifelse(data[["A"]] == 1, 1 / data$ps_lasso_clamped, 1 / (1 - data$ps_lasso_clam
>
>
> print("min and max weights")
```

```
[1] "min and max weights"
```

```
> print(min(data$weights_lasso))
```

```
[1] 1.062349
```

```
> print(max(data$weights_lasso))
```

```
[1] 8.964196
```

```
> #get formula
> print("formula")
```

```
[1] "formula"
```

```
> lasso_a_coefs <- coef(model_ps_lasso, s = "lambda.min")
> lasso_a_vars <- rownames(lasso_a_coefs)[which(as.matrix(lasso_a_coefs) != 0)]
> cov_lasso_a <- setdiff(lasso_a_vars, c("(Intercept)"))
> formula_lasso_ps_text <- paste("A ~", paste(cov_lasso_a, collapse = " + "))
> formula_lasso_ps_text_anotherver <- lasso_a_vars[which(lasso_a_vars != "(Intercept)")]
> print(formula_lasso_ps_text)
```

```
[1] "A ~ SEX1 + EDUCATION3 + EDUCATION5 + AGE + PAY_04 + PAY_AMT6"
```

```
> print(cov_lasso_a)
```

```
[1] "SEX1"       "EDUCATION3" "EDUCATION5" "AGE"        "PAY_04"
[6] "PAY_AMT6"
```

```
> # Estimate ACE using the weighted average
> #filter to only people who were enrolled A=1 or A=0
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
> # Filter to A0 or A1 groups, get default and multiply by weight
> EY_a0_ipw_lasso <- sum(as.numeric(as.character(a0_group$Y))* a0_group$weights_lasso)/sum(a0_group$weig
> EY_a1_ipw_lasso <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_lasso)/sum(a1_group$wei
>
>
> ace_ipw_lasso <- EY_a1_ipw_lasso - EY_a0_ipw_lasso
> print(paste("IPW ACE (Lasso):", round(ace_ipw_lasso,4)))
```

```
[1] "IPW ACE (Lasso): 0.1783"
```

```
> #see how weights are distr A=1 vs A=0
> data %>%
+  group_by(A) %>%
+  summarise(
+    Count = n(),
+    Min     = min(weights_lasso, na.rm = TRUE),
+    Average = mean(weights_lasso, na.rm = TRUE),
+    Median  = median(weights_lasso, na.rm = TRUE),
+    Perc_25 = quantile(weights_lasso, 0.25, na.rm = TRUE),
+    Perc_75 = quantile(weights_lasso, 0.75, na.rm = TRUE),
+    Max     = max(weights_lasso, na.rm = TRUE),
+    na_count = sum(is.na(weights_lasso)),
+    inf_count = sum(is.infinite(weights_lasso))
+  )
```

```
# A tibble: 2 x 10
  A     Count   Min Average Median Perc_25 Perc_75   Max na_count inf_count
  <fct> <int> <dbl>   <dbl>  <dbl>   <dbl>   <dbl> <dbl>    <int>     <int>
1 0       279  1.16    1.71   1.49    1.33    1.82  8.96        0         0
2 1       221  1.06    2.18   1.90    1.40    2.64  6.03        0         0
```

```
> #although max weight for A=0 is bigger, overall A=1 tends to have higher weights given perc_75, media
> #no NAs or inf
```

This shows the weights are more reasonable than the full dataset.

12 Lasso Variables, bootstrap

```
> bootstrap_results_lasso <- replicate(bootstrap_sample_size, {
+  #create bootstrap sample
+  sample_df <- data[sample(nrow(data), replace = TRUE), ]
+
+  #outcome regression
+
+  x_or_lasso_sample_df <- model.matrix(Y ~ A + ., data = sample_df[, c("Y", "A", cov_all)])[,-1]
+  y_or_lasso_sample_df <- sample_df$Y
+
+  col_idx_A1_sample_df <- which(colnames(x_or_lasso_sample_df) == "A1")
+  p_fac_or_sample_df <- rep(1, ncol(x_or_lasso_sample_df))
+  p_fac_or_sample_df[col_idx_A1_sample_df] <- 0
+
+  #fit lasso with cross-validation, get best lambda
+  model_or_lasso_sample_df <- cv.glmnet(x_or_lasso_sample_df, y_or_lasso_sample_df, family = "binomial
+  best_lambda_or_lasso_sample_df <- model_or_lasso_sample_df$lambda.min
+
+
+  x_all_enrolled_or_lasso_sample_df <- x_or_lasso_sample_df
+  x_all_enrolled_or_lasso_sample_df[, col_idx_A1_sample_df] <- 1  # Everyone treated
+  x_no_enrolled_or_lasso_sample_df <- x_or_lasso_sample_df
+  x_no_enrolled_or_lasso_sample_df[, col_idx_A1_sample_df] <- 0  # Everyone untreated
+
+  #now predict
+  sample_df_prob_Y_if_A1 <- as.numeric(predict(model_or_lasso_sample_df, s = best_lambda_or_lasso_samp
+  EY_a1_or_lasso_sample_df <- mean(sample_df_prob_Y_if_A1)
+
+
+  sample_df_prob_Y_if_a0 <- as.numeric(predict(model_or_lasso_sample_df, s = best_lambda_or_lasso_samp
+  EY_a0_or_lasso_sample_df <- mean(sample_df_prob_Y_if_a0)
+
+
+  ace_or_lasso_sample_df <- EY_a1_or_lasso_sample_df - EY_a0_or_lasso_sample_df
+
+
+   #IPW
+
+
+  #prepare data
+  x_ps_lasso_sample_df <- model.matrix(A ~ ., data = sample_df[, c("A", cov_all)])[,-1]
+  y_ps_lasso_sample_df <- sample_df$A
+
+
+  #fit lasso
+  model_ps_lasso_sample_df <- cv.glmnet(x_ps_lasso_sample_df, y_ps_lasso_sample_df, family = "binomial
+  best_lambda_ps_lasso_sample_df <- model_ps_lasso_sample_df$lambda.min
+  #get propensity scores
+  sample_df$ps_lasso_sample_df <- as.numeric(predict(model_ps_lasso_sample_df, s = best_lambda_ps_lasso
+
+
+  sample_df$ps_lasso_clamped_sample_df <- pmax(pmin(sample_df$ps_lasso_sample_df, 0.9999), 0.0001)
```

```
+  # calculate weights.
+  sample_df$weights_lasso_sample_df <- ifelse(sample_df[["A"]] == 1, 1 / sample_df$ps_lasso_clamped_sa
+
+
+  # Estimate ACE using the weighted average
+  #filter to only people who were enrolled A=1 or A=0
+  a0_group_sample_df <- sample_df[sample_df$A == 0, ]
+  a1_group_sample_df <- sample_df[sample_df$A == 1, ]
+  # Filter to A0 or A1 groups, get default and multiply by weight
+  EY_a0_ipw_lasso_sample_df <- sum(as.numeric(as.character(a0_group_sample_df$Y)) * a0_group_sample_df
+  EY_a1_ipw_lasso_sample_df <- sum(as.numeric(as.character(a1_group_sample_df$Y)) * a1_group_sample_df
+
+
+  ace_ipw_lasso_sample_df <-  EY_a1_ipw_lasso_sample_df - EY_a0_ipw_lasso_sample_df
+
+
+  return(c(ace_or_lasso_sample_df, ace_ipw_lasso_sample_df))
+ })
```

```
> orig_ace_lasso <- c(ace_or_lasso, ace_ipw_lasso)
>
>
> # convert to dataframe
> boot_df_lasso <- as.data.frame(t(bootstrap_results_lasso))
> colnames(boot_df_lasso) <- c("Regression", "IPW")
>
>
> # Calculate Metrics
> metrics_lasso <- data.frame(
+  Method = c("Regression", "IPW"),
+  Original_ACE = orig_ace_lasso, # Use values from the full dataset
+  Bootstrap_ACE = c(mean(boot_df_lasso$Regression), mean(boot_df_lasso$IPW)),
+  Standard_Error = c(sd(boot_df_lasso$Regression), sd(boot_df_lasso$IPW))
+ )
>
>
> # Calculate Bias
> metrics_lasso$Bootstrap_Bias <- metrics_lasso$Bootstrap_ACE - metrics_lasso$Original_ACE
> print(metrics_lasso)
```

```
      Method Original_ACE Bootstrap_ACE Standard_Error Bootstrap_Bias
1 Regression    0.2032658     0.2044508     0.04012864    0.001185043
2        IPW    0.1782535     0.1952790     0.04284862    0.017025427
```

This suggests that things are more stable with the lasso.

13 3 Now, do rank PC analysis!!

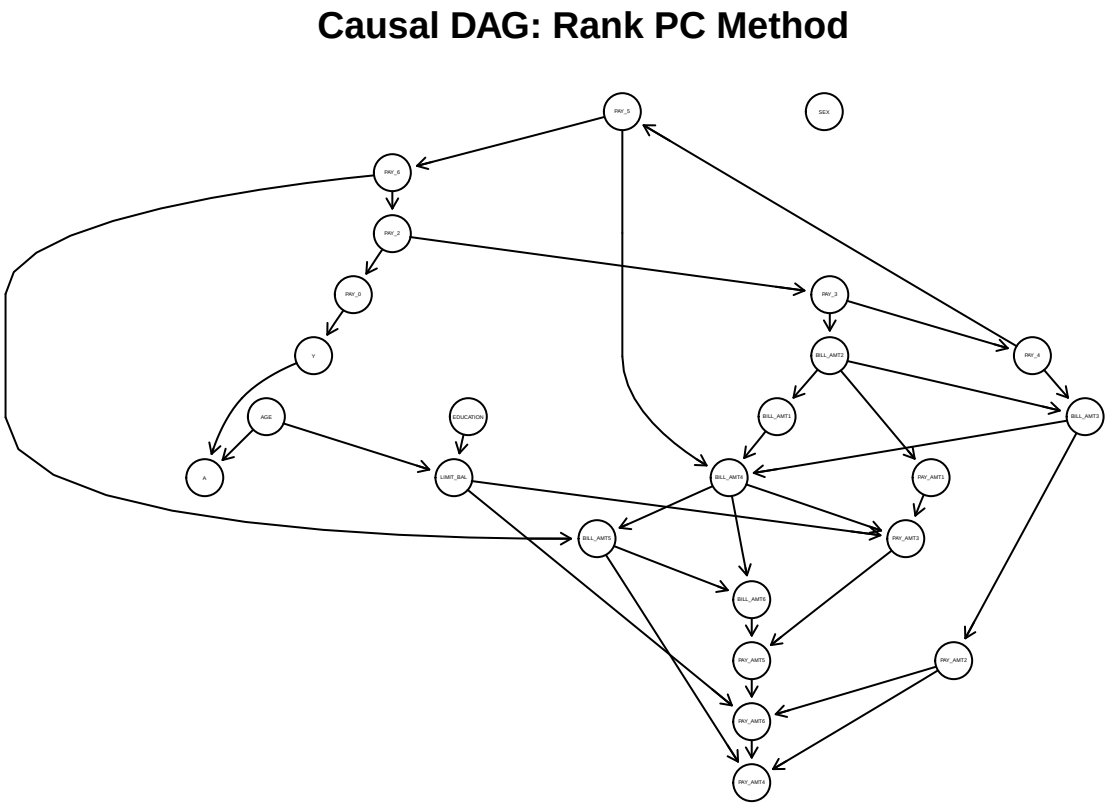
PC requires data follow Gaussian distribution and have linear relationships - that's why we transformed the variables. The "Rank PC" method aims to make the PC algorithm robust to non-normal distributions and non-linear relationships.

PC systematically tests for conditional independence between variables to determine if there should be an edge - gaussian conditional independence test, assuming variables normally distributed.

Rank PC uses rank based test, say using spearman's rank correlation (considers rank of value not raw values). Does not look at raw values - looks at rank of the value relative to others. Does not assume a specific distribution. Somewhat robust to outliers, because just ranking. On the other hand, if data is not evenly spread, could misrepresent? Because it treats gaps of 100, 101, and 102 as the same as 100, 5k, 100k.

14 Non-transformed, Rank PC

```
> #convert data to numerical for gaussian tests
> rank_pc_data <- data %>%
+   mutate(across(where(is.factor), ~as.numeric(as.factor(.x)))) %>% #rank my factors into numbers
+   select(Y, A, all_of(cov_all))
>
> # calculate correlation matrix and sample size; use to test for conditional independence amongst pair
> rank_pc_corr_size <- list(C = cor(rank_pc_data, method = "spearman"), n = nrow(rank_pc_data))
>
> # Run PC; alpha serves as a tuning parameter for the significance of independence tests
> rank_pc_fit <- pc(suffStat = rank_pc_corr_size,
+                   indepTest = gaussCItest,
+                   labels = colnames(rank_pc_data),
+                   alpha = 0.01,
+                   verbose = FALSE)
>
> # Plot DAG
> plot(rank_pc_fit@graph, main = "Causal DAG: Rank PC Method")
```




```
> #export
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_rank_pc_method.png", width = 1200, height = 1000, r
>
```

```
> plot(rank_pc_fit@graph, main = "Causal DAG: Rank PC Method")
>
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> rank_pc_dag <- pcalg2dagitty(as(rank_pc_fit@graph, "matrix"),
+                         labels = colnames(rank_pc_data),
+                         type = "cpdag")
>
> # Get the Markov Blanket (Parents + Children + Spouses)
> rank_pc_mb_y <- markovBlanket(rank_pc_dag, "Y")
> print(rank_pc_mb_y)
```

```
[1] "A"     "PAY_0"
```

```
> # Update your covariate list for the models
> cov_rank_pc_y <- setdiff(rank_pc_mb_y, "A") #exclude A
> print(cov_rank_pc_y) #only PAY_0?
```

```
[1] "PAY_0"
```

```
> # Get the Markov Blanket (Parents + Children + Spouses)
> rank_pc_mb_a <- markovBlanket(rank_pc_dag, "A")
> print(rank_pc_mb_a)
```

```
[1] "AGE"       "Y"         "LIMIT_BAL"
```

```
> # Update your covariate list for the models
> cov_rank_pc_a <- setdiff(rank_pc_mb_a, "Y") #exclude A
> print(cov_rank_pc_a) #only PAY_0?
```

```
[1] "AGE"       "LIMIT_BAL"
```

rank pc algorithm nontransformed or

```
``` r
> #outcome regression
> if(length(cov_rank_pc_y) == 0) {
+   formula_or_rank_pc <- as.formula("Y ~ A")
+ } else {
+   formula_or_rank_pc <- as.formula(paste("Y ~ A +", paste(cov_rank_pc_y, collapse = "+")))
+ }
> model_or_rank_pc <- glm(formula_or_rank_pc, data = data, family = binomial)
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_or_rank_pc)
```

```
Y ~ A + PAY_0
```

```
> # pc_data_all_enrolled <- data_all_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
```
```

```
> #   select(Y, A, all_of(cov_all))
>
> # pc_data_no_enrolled <- data_no_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
> #   select(Y, A, all_of(cov_all))
>
> # Predict default probability for world of everyone enrolled, and no one enrolled
> data$prob_Y_if_A1_rank_pc_var<- predict(model_or_rank_pc, newdata = data_all_enrolled, type = "respons
>
> EY_a1_or_rank_pc_var <- mean(data$prob_Y_if_A1_rank_pc_var)
>
> data$prob_Y_if_A0_rank_pc_var <- predict(model_or_rank_pc, newdata = data_no_enrolled, type = "respons
> EY_a0_or_rank_pc_var <- mean(data$prob_Y_if_A0_rank_pc_var)
>
> ace_or_rank_pc_var <- EY_a1_or_rank_pc_var - EY_a0_or_rank_pc_var
>
> #print means
> print("Default likelihood (Rank PC Set, OR), A=1 then A=0")
```

```
[1] "Default likelihood (Rank PC Set, OR), A=1 then A=0"
```

```
> print(EY_a1_or_rank_pc_var)
```

```
[1] 0.4284831
```

```
> print(EY_a0_or_rank_pc_var)
```

```
[1] 0.2210747
```

```
> print(paste("OR ACE (Rank PC Set):", round(ace_or_rank_pc_var,4)))
```

```
[1] "OR ACE (Rank PC Set): 0.2074"
```

15 Rank PC algorithm nonttransformed ipw

```
> if(length(cov_rank_pc_a) == 0) {
+   formula_ps_rank_pc_var <- as.formula("A ~ 1")
+ } else {
+   formula_ps_rank_pc_var <- as.formula(paste("A~", paste(cov_rank_pc_a, collapse = "+"))) #formula of
+ }
> #get propensity scores
> model_ps_rank_pc_var <- glm(formula_ps_rank_pc_var, data = data, family = binomial) #binomial because
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_ps_rank_pc_var)
```

```
A ~ AGE + LIMIT_BAL
```

```
> data$ps_rank_pc_var <- predict(model_ps_rank_pc_var, type = "response") #calculate propensity scores
> print("min and max ps")
```

```
[1] "min and max ps"
```

```
> print(min(data$ps_rank_pc_var))
```

```
[1] 0.1102271
```

```
> print(max(data$ps_rank_pc_var))
```

```
[1] 0.9688367
```

```
> #extreme ps means likely have extreme scores
>
> data$ps_rank_pc_var_clamped <- pmax(pmin(data$ps_rank_pc_var, 0.9999), 0.0001)
>
>
> # calculate weights.
> data$weights_rank_pc_var <- ifelse(data[["A"]] == 1, 1 / data$ps_rank_pc_var_clamped, 1 / (1 - data$ps
>
> print("min and max weights")
```

```
[1] "min and max weights"
```

```
> print(min(data$weights_rank_pc_var)) #1.0001
```

```
[1] 1.032166
```

```
> print(max(data$weights_rank_pc_var)) #2.8
```

```
[1] 9.244483
```

```
> #if customer was actually enrolled (A==1), then d weight as 1/ps, otherwise do 1/(1-ps)
> #So if customer had A=1 & ps=0.8, then weight = 1.25; but if A=0 then weight=5. Scales by unexpectedn
>
> # Estimate ACE using the weighted average
> #filter to only people who were enrolled A=1 or A=0
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
> # Filter to A0 or A1 groups, get default and multiply by weight; basically, if paid on time, contribu
> EY_a0_ipw_rank_pc_var <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_rank_pc_var)/sum
> EY_a1_ipw_rank_pc_var <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_rank_pc_var)/sum
>
> ace_ipw_rank_pc_var <- EY_a1_ipw_rank_pc_var - EY_a0_ipw_rank_pc_var
> print(paste("IPW ACE (Rank PC set):", round(ace_ipw_rank_pc_var,4)))
```

```
[1] "IPW ACE (Rank PC set): 0.184"
```

16 Rank Pc algorithm bootstrap nontransformed

```
> bootstrap_results_rank_pc <- replicate(bootstrap_sample_size, {
+   #bootstrap sample
+   sample_df <- data[sample(nrow(data), replace = TRUE), ]
+
+   #create pc markov blanket
+   boot_data <- sample_df %>%
+   mutate(across(where(is.factor), ~as.numeric(as.factor(.x)))) %>% #rank my factors into numbers
+   select(Y, A, all_of(cov_all))
+
+   #get correlation, sample size of bootstrapped data
```

```
+   suff_boot <- list(C = cor(boot_data, method = "spearman"), n = nrow(boot_data))
+
+   #Run rank PC Algorithm
+   fit_boot <- pc(suffStat = suff_boot, indepTest = gaussCItest,
+                  labels = colnames(boot_data), alpha = 0.01, verbose = FALSE)
+
+   adj_matrix <- as(fit_boot@graph, "matrix")
+   adj_matrix_input <- pcalg2dagitty(adj_matrix,
+                        labels = colnames(boot_data),
+                        type = "cpdag")
+
+   # Get the Markov Blanket (Parents + Children + Spouses)
+   y_indices_all_boot <- markovBlanket(adj_matrix_input, "Y")
+   cov_boot <- setdiff(y_indices_all_boot, "A") #exclude A
+
+   a_indices_all_boot <- markovBlanket(adj_matrix_input, "A")
+   cov_boot_a <- setdiff(a_indices_all_boot, "Y")
+
+   #what is markov blanket empty?
+   if(length(cov_boot) == 0) {
+     formula_or_boot <- as.formula("Y ~ A")
+   } else {
+     formula_or_boot <- as.formula(paste("Y ~ A +", paste(cov_boot, collapse = "+")))
+   }
+   if(length(cov_boot_a) == 0) {
+     formula_ps_boot <- as.formula("A ~ 1")
+   } else {
+     formula_ps_boot <- as.formula(paste("A ~", paste(cov_boot_a, collapse = "+")))
+   }
+
+   # outcome regression
+   model_or_boot <- glm(formula_or_boot, data = sample_df, family = binomial)
+
+   #create fake datasets
+   sample_df_a0 <- sample_df
+   sample_df_a0$A <- factor(0, levels = levels(sample_df$A))
+   sample_df_a1 <- sample_df
+   sample_df_a1$A <- factor(1, levels = levels(sample_df$A))
+
+   # Predict default probability for world of everyone enrolled, and no one enrolled
+   sample_df_prob_Y_if_A1 <- predict(model_or_boot, newdata = sample_df_a1, type = "response")
+   EY_a1_or_sample_df <- mean(sample_df_prob_Y_if_A1)
+
+   sample_df_prob_Y_if_A0 <- predict(model_or_boot, newdata = sample_df_a0, type = "response")
+   EY_a0_or_sample_df <- mean(sample_df_prob_Y_if_A0)
+
+   ace_or_sample_df <- EY_a1_or_sample_df - EY_a0_or_sample_df
+
+   #IPW
+   #get propensity score
+   model_ps_sample_df <- glm(formula_ps_boot, data = sample_df, family = binomial)
+   sample_df$ps_sample_df <- predict(model_ps_sample_df, type = "response")
+   #clamp ps score between 0.0001 and 0.9999 because otherwise get warnings of fitted prob being 0 or
```

```
+   sample_df$ps_sample_df_clamped <- pmax(pmin(sample_df$ps_sample_df, 0.9999), 0.0001)
+   #get weights
+   sample_df$weights_sample_df <- ifelse(sample_df$A == 1, 1/sample_df$ps_sample_df_clamped, 1/(1-sampl
+   #divide into a0 and a1 groups
+   a0_group_sample_df <- sample_df[sample_df$A == 0, ]
+   a1_group_sample_df <- sample_df[sample_df$A == 1, ]
+   #calculate ACE
+   EY_a0_ipw_sample_df <- sum(as.numeric(as.character(a0_group_sample_df$Y)) * a0_group_sample_df$weigh
+   EY_a1_ipw_sample_df <- sum(as.numeric(as.character(a1_group_sample_df$Y)) * a1_group_sample_df$weigh
+   ace_ipw_sample_df <-  EY_a1_ipw_sample_df - EY_a0_ipw_sample_df
+
+   return(list(ace_or = ace_or_sample_df,
+               ace_ipw = ace_ipw_sample_df,
+               adj_matrix = adj_matrix,
+               blanket = y_indices_all_boot,
+               blanket_a = a_indices_all_boot
+               ))
+ }, simplify = FALSE)
```

```
> orig_ace_rank_pc_var <- c(ace_or_rank_pc_var, ace_ipw_rank_pc_var)
>
> #extract ACE values
> boot_aces_or_rank_pc  <- unlist(lapply(bootstrap_results_rank_pc, function(x) x$ace_or))
> boot_aces_ipw_rank_pc <- unlist(lapply(bootstrap_results_rank_pc, function(x) x$ace_ipw))
>
> #extract markov blanket
> boot_blankets_rank_pc <- lapply(bootstrap_results_rank_pc, function(x) x$blanket)
> boot_blankets_rank_pc_a <- lapply(bootstrap_results_rank_pc, function(x) x$blanket_a)
>
> # convert to dataframe
> boot_df_rank_pc_var <- data.frame(
+   Regression = boot_aces_or_rank_pc,
+   IPW = boot_aces_ipw_rank_pc
+ )
> colnames(boot_df_rank_pc_var) <- c("Regression", "IPW")
>
> # Calculate Metrics
> metrics_rank_pc_var <- data.frame(
+   Method = c("Regression", "IPW"),
+   Original_ACE = orig_ace_rank_pc_var, # Use values from the full dataset
+   Bootstrap_ACE = c(mean(boot_df_rank_pc_var$Regression), mean(boot_df_rank_pc_var$IPW)),
+   Standard_Error = c(sd(boot_df_rank_pc_var$Regression), sd(boot_df_rank_pc_var$IPW))
+ )
>
> # Calculate Bias
> metrics_rank_pc_var$Bootstrap_Bias <- metrics_rank_pc_var$Bootstrap_ACE - metrics_rank_pc_var$Original
> print(metrics_rank_pc_var)
```

```
      Method Original_ACE Bootstrap_ACE Standard_Error Bootstrap_Bias
1 Regression    0.2074085     0.2028526     0.04123487   -0.004555925
2        IPW    0.1839686     0.1858427     0.04571872    0.001874059
```

```
> boot_blankets_rank_pc_unlisted <- unlist(boot_blankets_rank_pc)
>
```

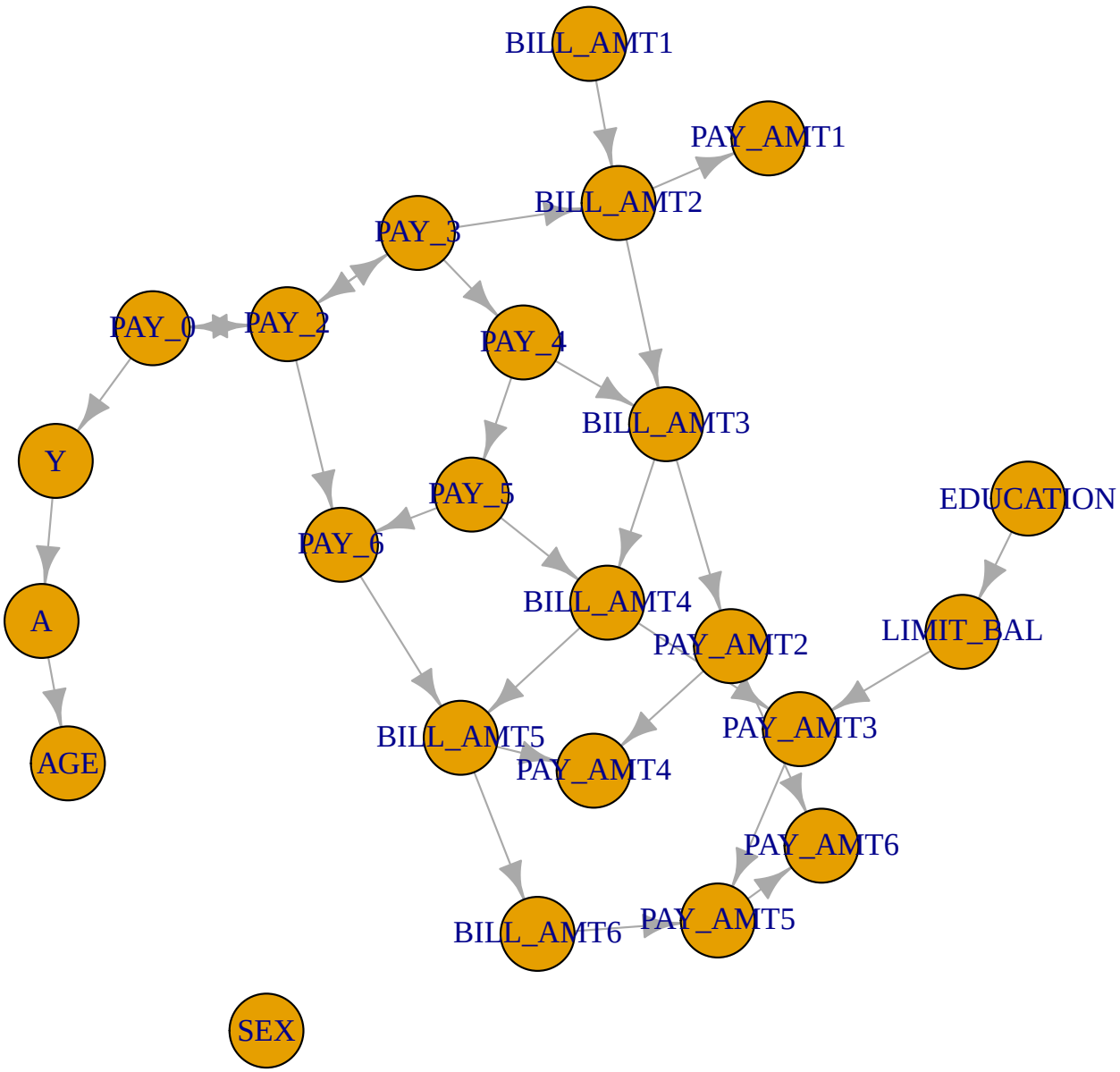
```
> boot_mb_stability_rank_pc <- table(boot_blankets_rank_pc_unlisted) / bootstrap_sample_size
> print("Selection Frequency for Y's Markov Blanket:")

[1] "Selection Frequency for Y's Markov Blanket:"
> print(sort(boot_mb_stability_rank_pc, decreasing = TRUE))

boot_blankets_rank_pc_unlisted
    PAY_0         A     PAY_3     PAY_2       AGE     PAY_5     PAY_6  PAY_AMT2
    0.919     0.912     0.303     0.216     0.114     0.094     0.061     0.060
 PAY_AMT4     PAY_4 LIMIT_BAL  PAY_AMT3  PAY_AMT1 EDUCATION       SEX
    0.016     0.015     0.012     0.011     0.009     0.004     0.001
> boot_blankets_rank_pc_unlisted_a <- unlist(boot_blankets_rank_pc_a)
>
> boot_mb_stability_rank_pc_a <- table(boot_blankets_rank_pc_unlisted_a) / bootstrap_sample_size
> print("Selection Frequency for A's Markov Blanket:")

[1] "Selection Frequency for A's Markov Blanket:"
> print(sort(boot_mb_stability_rank_pc_a, decreasing = TRUE))

boot_blankets_rank_pc_unlisted_a
        Y       AGE EDUCATION LIMIT_BAL     PAY_3     PAY_0  PAY_AMT2  PAY_AMT4
    0.912     0.906     0.381     0.226     0.148     0.106     0.046     0.031
      SEX  PAY_AMT3  PAY_AMT6  PAY_AMT1     PAY_4
    0.018     0.011     0.011     0.006     0.001
> #get adjacent matrices
> all_adj_bootstrap_rank_pc <- lapply(bootstrap_results_rank_pc, function(x) x$adj_matrix)
> #sum them up to get edge frequencies - prob an edge exists between 2 variables
> consensus_matrix_bootstrap_rank_pc <- Reduce("+", all_adj_bootstrap_rank_pc) / bootstrap_sample_size
>
> # Create a graph where an edge exists if it appeared in > 50% of samples
> average_adj_bootstrap_rank_pc <- (consensus_matrix_bootstrap_rank_pc > 0.5) * 1
>
> # Convert back to a graph object for plotting
> avg_graph_bootstrap_rank_pc <- graph_from_adjacency_matrix(average_adj_bootstrap_rank_pc, mode = "dir
> plot(avg_graph_bootstrap_rank_pc, main = "Consensus DAG (Edges in >50% of Bootstraps)")
```




```
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/consensus_dag_rank_pc_bootstrap.png", width = 1200, height = 1
> par(mar = c(1, 1, 3, 1))
> plot(avg_graph_bootstrap_rank_pc,
+      main = "Consensus DAG (Edges in >50% of Bootstraps): Rank PC Method",
+      vertex.size = 15,              # Smaller nodes
+      vertex.label.cex = 0.6,        # Smaller labels
+      vertex.label.dist = 1.2,       # Offset labels from node centers
+      edge.arrow.size = 0.3,         # Smaller arrows
+      edge.curved = 0.2)             # Slight curve to distinguish overlapping arrows
> dev.off()
```

pdf

2

17 4 Now the Analysis, for PC-selected variables! (limited vars to most recent month first)

18 PC Algorithm

Find the markov blanket of Y, consisting of its parents, children, and parents of its children. Y is conditionally independent of all other variables GIVEN its Markov Blanket.

So: use PC algorithm to find the full DAG. Identify variables directly connected to Y. Run OR and IPW using the variables directly connected to Y.

Note: assumes 1. Causal sufficiency - no hidden confounders 2. faithfulness: conditional independencies in the data imply cond ind in the DAG. 3. Markov condition: variable is conditionally indpendent of its non-descendents given its parents, in the DAG.

PC measures cond ind in the data, and searches for a graph to satisfy them.

1. start with fully connected, UNDIRECTED graph
2. test for conditional independence amongst all pairs nad subsets of variables –> X ind of Y given Z, for different sets of Z
3. remove edges based on these tests
4. continue until we can't remove more edges without violating conditional independences in data
5. apply orientation rules (colliders etc) to orient undirected edges, as possible
6. adjust to make sure graph remains acyclic

For Gaussian data, this is like testing if the null hypothesis is rho of X&Y given Z is 0, vs not. If data is categorical, can test conditional independence using a chi-square test.

Obtain the DAG.

The "Penalty" of Categorical Data

When you use a Gaussian test (like gaussCItest), the algorithm treats everything as a simple linear correlation. It only needs to find one "slope" to connect two variables.

mixCItest is different. When it tests a factor (like EDUCATION with 4 levels) against your outcome Y: It essentially creates a contingency table or a grouped model. If you only have 500 rows, the data gets split into very small "buckets" (e.g., how many people with "Graduate School" education actually "Defaulted"?). If those buckets are too small, the p-value becomes very large (insignificant), and the algorithm deletes the line. 2. Loss of Power in Conditional Tests The PC algorithm doesn't just test A vs B. It tests A vs B given C, D, and E. As the "Conditioning Set" grows, mixCItest has to slice your 500 rows even thinner. It is much harder to prove that LIMIT_BAL and Y are dependent after accounting for PAY_0 and EDUCATION when using a mixed-type test. The test "gives up" more easily than a purely numeric one would. 3. "Conservative" by Design The micd package (which provides mixCItest) is designed to avoid False Positives. It would rather tell you two variables are independent than show a "fake" causal link. In biostatistics and high-stakes financial modeling, a "Sparse" graph is often considered more trustworthy than a "Spaghetti" graph where everything is connected to everything.

```
> pc_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
> # pc_data_var_types <- ifelse(sapply(pc_data, is.factor), 1, 0)
> print(summary(pc_data)) #proves data is in right format.
```

```
 Y       A         LIMIT_BAL      SEX     EDUCATION      AGE
 0:344   0:279   Min.   : 10000   0:291   1:176     Min.   :21.00
```

```
 1:156   1:221   1st Qu.: 50000   1:209   2:241     1st Qu.:28.00
                 Median :140000           3: 74     Median :34.00
                 Mean   :164060           4:  2     Mean   :35.35
                 3rd Qu.:230000           5:  7     3rd Qu.:40.00
                 Max.   :620000                     Max.   :65.00

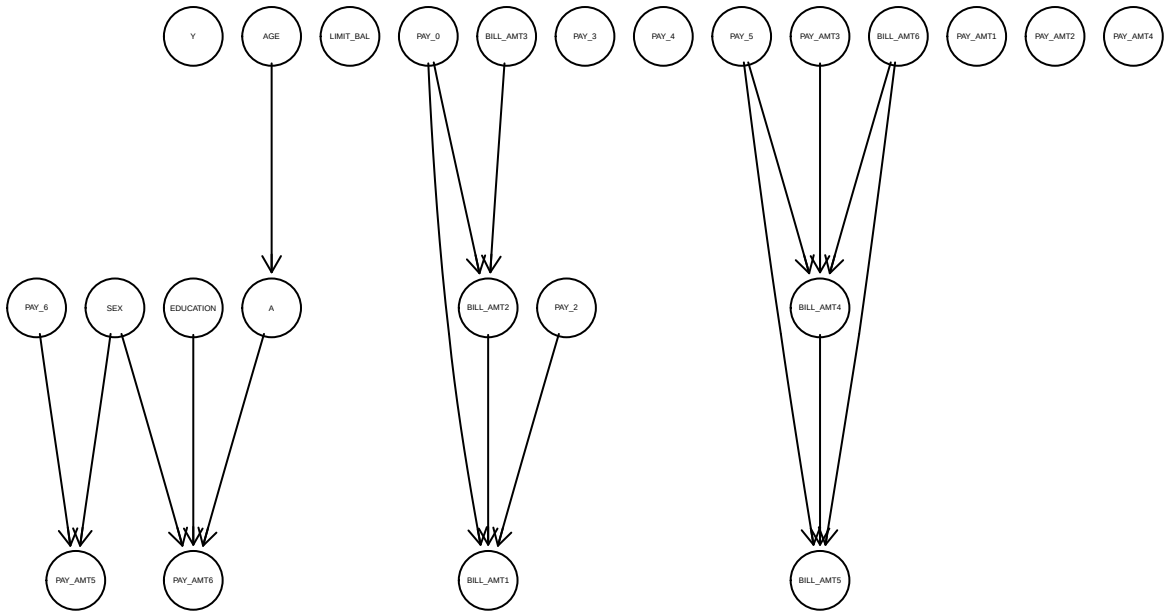
    PAY_0     PAY_2    PAY_3    PAY_4    PAY_5    PAY_6      BILL_AMT1
0      :232   -2: 64   -2: 68   -2: 74   -2: 72   -2: 76   Min.   :  -532
-1     :100   -1:109   -1:115   -1:106   -1: 99   -1:102   1st Qu.:  3744
1      : 71   0 :255   0 :255   0 :248   0 :270   0 :265   Median : 22156
-2     : 48   2 : 63   2 : 57   2 : 69   2 : 56   2 : 54   Mean   : 49072
2      : 42   3 :  5   3 :  4   3 :  3   3 :  2   3 :  3   3rd Qu.: 58694
3      :  4   4 :  3   4 :  1            4 :  1            Max.   :604019
(Other):  3   5 :  1
  BILL_AMT2        BILL_AMT3        BILL_AMT4        BILL_AMT5
Min.   :-17710   Min.   :-17706   Min.   : -2800   Min.   :-81334
1st Qu.:  2418   1st Qu.:  2583   1st Qu.:  2888   1st Qu.:  2451
Median : 19870   Median : 19507   Median : 18654   Median : 18538
Mean   : 45531   Mean   : 42682   Mean   : 40983   Mean   : 38318
3rd Qu.: 55343   3rd Qu.: 51370   3rd Qu.: 49070   3rd Qu.: 45385
Max.   :605943   Max.   :467911   Max.   :404157   Max.   :587067

  BILL_AMT6         PAY_AMT1         PAY_AMT2         PAY_AMT3
Min.   : -2506   Min.   :     0   Min.   :     0   Min.   :     0.0
1st Qu.:  1228   1st Qu.:  1000   1st Qu.:  1000   1st Qu.:   498.2
Median : 17972   Median :  2000   Median :  2038   Median :  1737.5
Mean   : 36065   Mean   :  4940   Mean   :  5766   Mean   :  5872.4
3rd Qu.: 44432   3rd Qu.:  4753   3rd Qu.:  5000   3rd Qu.:  4906.8
Max.   :323821   Max.   :234000   Max.   :294318   Max.   :319494.0

   PAY_AMT4         PAY_AMT5         PAY_AMT6
Min.   :     0   Min.   :     0   Min.   :     0.0
1st Qu.:   298   1st Qu.:   390   1st Qu.:   181.5
Median :  1576   Median :  1529   Median :  1500.0
Mean   :  4258   Mean   :  5212   Mean   :  5084.9
3rd Qu.:  3956   3rd Qu.:  4000   3rd Qu.:  4134.0
Max.   :109332   Max.   :331788   Max.   :189600.0
```

```
> # attr(pc_data, "adaptDF") <- TRUE
>
> # Run PC; alpha serves as a tuning parameter for the significance of independence tests
> pc_fit_test <- pc(suffStat = pc_data,
+                   indepTest = mixCItest,
+                   labels = colnames(pc_data),
+                   alpha = 0.01,
+                   verbose = FALSE,
+                   u2pd = "relaxed",
+                   skel.method = "stable" ,  #order of test doesnt matter
+                   maj.rule = FALSE, #if some say a-->b and some say b-->a, then pick direction with
+                   solve.confl = FALSE #if some say a-->b and some say b-->a, then pick a direction
+                   )
>
> # Plot DAG
```

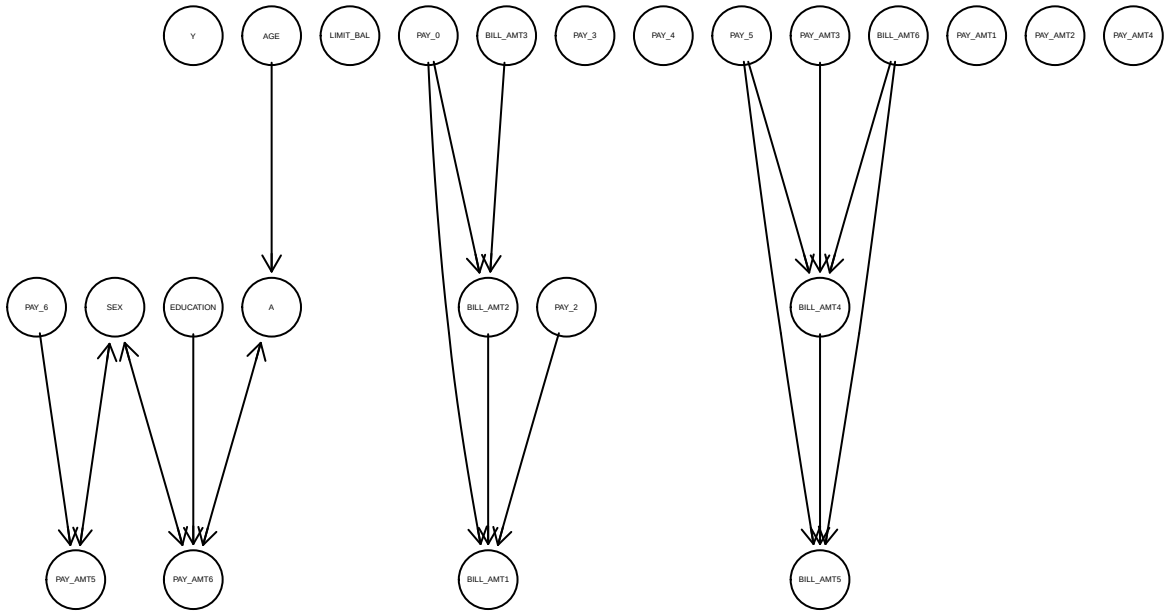
```
> plot(pc_fit_test@graph, main = "TEST Causal DAG: Markov Blanket Discovery (PC Method)")
```




```
> #export
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_test.png", width = 1200, height = 1000, r
>
> plot(pc_fit_test@graph, main = "TEST Causal DAG: Markov Blanket Discovery (PC Method)")
>
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

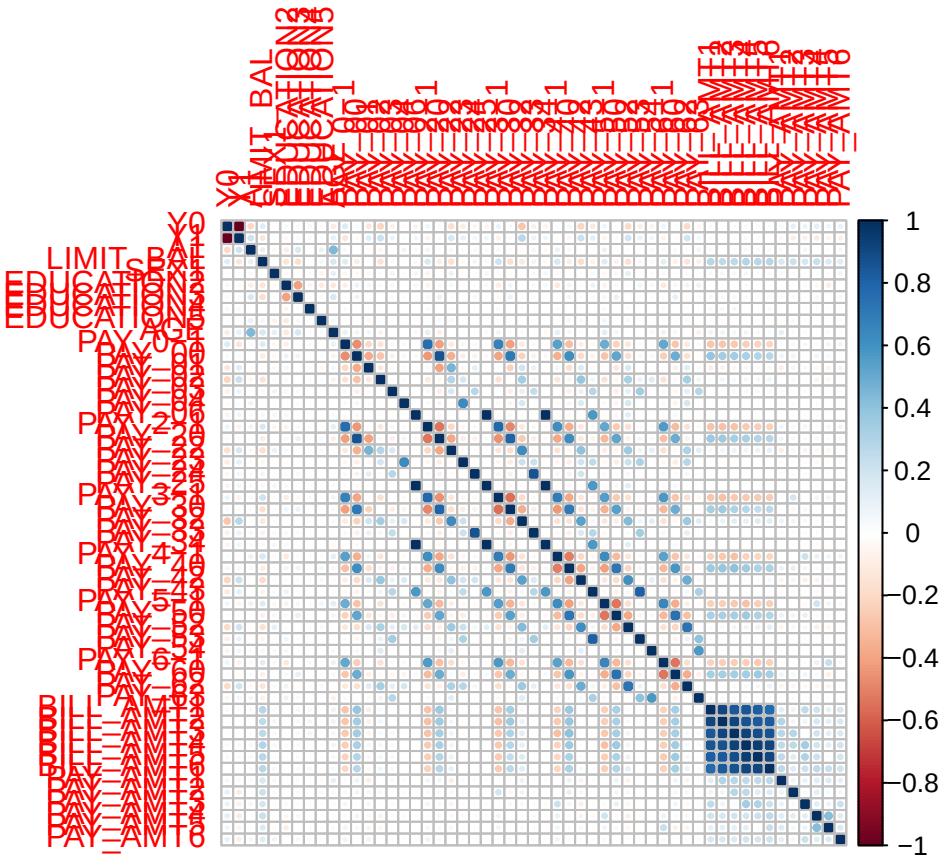
```
> # Run PC; alpha serves as a tuning parameter for the significance of independence tests
> pc_fit_test <- pc(suffStat = pc_data,
+                   indepTest = mixCItest,
+                   labels = colnames(pc_data),
+                   alpha = 0.01,
+                   verbose = FALSE,
+                   u2pd = "relaxed",
+                   skel.method = "stable" ,  #order of test doesnt matter
+                   maj.rule = FALSE,
+                   solve.confl = TRUE #if some say a-->b and some say b-->a, then pick a direction
+                   )
>
> # Plot DAG
> plot(pc_fit_test@graph, main = "TEST Causal DAG: Markov Blanket Discovery (PC Method)")
```




```
> #export
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_v2_test.png", width = 1200, height = 1000
>
> plot(pc_fit_test@graph, main = "TEST Causal DAG: Markov Blanket Discovery (PC Method)")
>
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> corrplot(cor(model.matrix(~.-1, data=pc_data)), method="circle")
```




```
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/correlation_matrix_default_test.png", width = 1000, height = 1
> corrplot(cor(model.matrix(~.-1, data=pc_data)), method="circle")
> dev.off()
```

```
pdf
  2
```

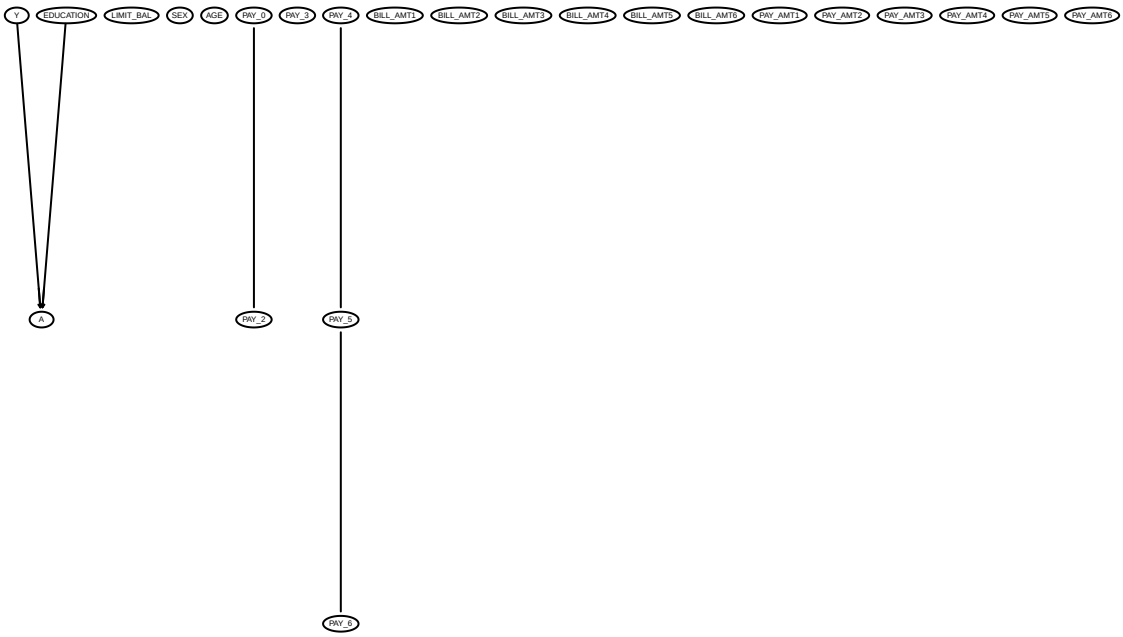
Note the correlation is really low, resulting in the PC algorithm not finding many edges. This is likely because of the categorical variables- the mixCItest is struggling to find significant relationships because of the small effective sample size when conditioning on multiple variables. This is not solved even when we limit to max conditioning set size of 2.

```
> #convert data to numerical for gaussian tests
> pc_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
>
> # calculate correlation matrix and sample size; use to test for conditional independence amongst pair
> # pc_corr_size <- list(C = cor(pc_data), n = nrow(pc_data))
>
>
> pc_fit_bnlearn <- pc.stable(pc_data,
+                     # blacklist = master_blacklist,
+                       # test = "mix", #not needed, will automatically detect which test based on vari
+                     alpha = 0.01)
>
>
> # Get adjacency matrix from PC fit
```

```
> # pc_fit_adj_matrix <- amat(pc_fit)
> # print(pc_fit_adj_matrix)
>
> pc_fit_bnlearn_cextend <- cextend(pc_fit_bnlearn) #directs edges as much as possible, but doesn't ori
>
>
> pc_fit_bnlearn_maxsx <- pc.stable(pc_data,
+                       # blacklist = master_blacklist,
+                         # test = "mix", #not needed, will automatically detect which test based on vari
+                       alpha = 0.01,
+                       max.sx=2) #limits size of conditioning set to 2, which should help with the power
> pc_fit_bnlearn_maxsx_cextend <- cextend(pc_fit_bnlearn_maxsx) #directs edges as much as possible, but
>
> cov_most_recent <- c("PAY_0", "EDUCATION", "AGE", "SEX", "PAY_AMT1", "BILL_AMT1", "LIMIT_BAL")
>
> pc_data_limitedcov <- pc_data %>%
+   select(Y, A, all_of(cov_most_recent))
> pc_fit_bnlearn_limitedcov <- pc.stable(pc_data_limitedcov,
+                       # blacklist = master_blacklist,
+                         # test = "mix", #not needed, will automatically detect which test based on vari
+                       alpha = 0.01)
```

```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```

TEST Full Causal DAG, PC Method



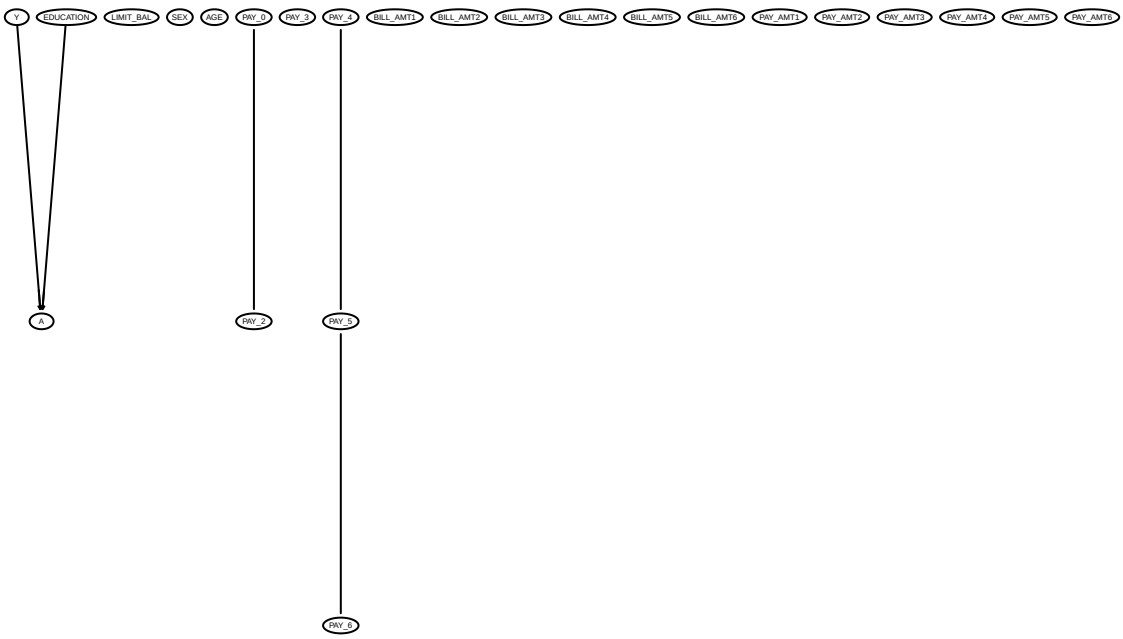

```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_recentmonth_test.png", width = 1600, heigh
> graphviz.plot(pc_fit_bnlearn, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```

```
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> # # Plot DAG
> # graphviz.plot(pc_fit_cextend, shape = "ellipse",
> #       main = "Full Causal DAG, PC Method")
>
> # # 2. Export a high-quality version
> # png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_cextend.png", width = 1600, hei
> # graphviz.plot(pc_fit_cextend, shape = "ellipse",
> #       main = "Full Causal DAG, PC Method")
> # # 3. Close the device to save the file
> # dev.off()
```

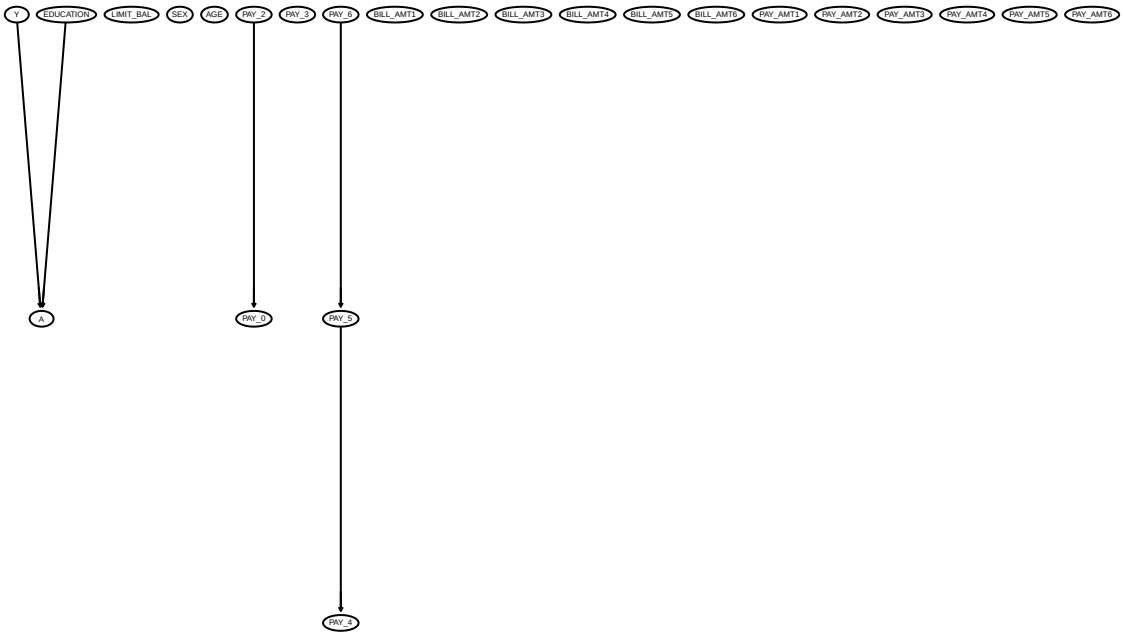
```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_maxsx, shape = "ellipse",
+       main = "Full Causal DAG, PC Method")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_maxsx_test.png", width = 1600, he
> graphviz.plot(pc_fit_bnlearn_maxsx, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

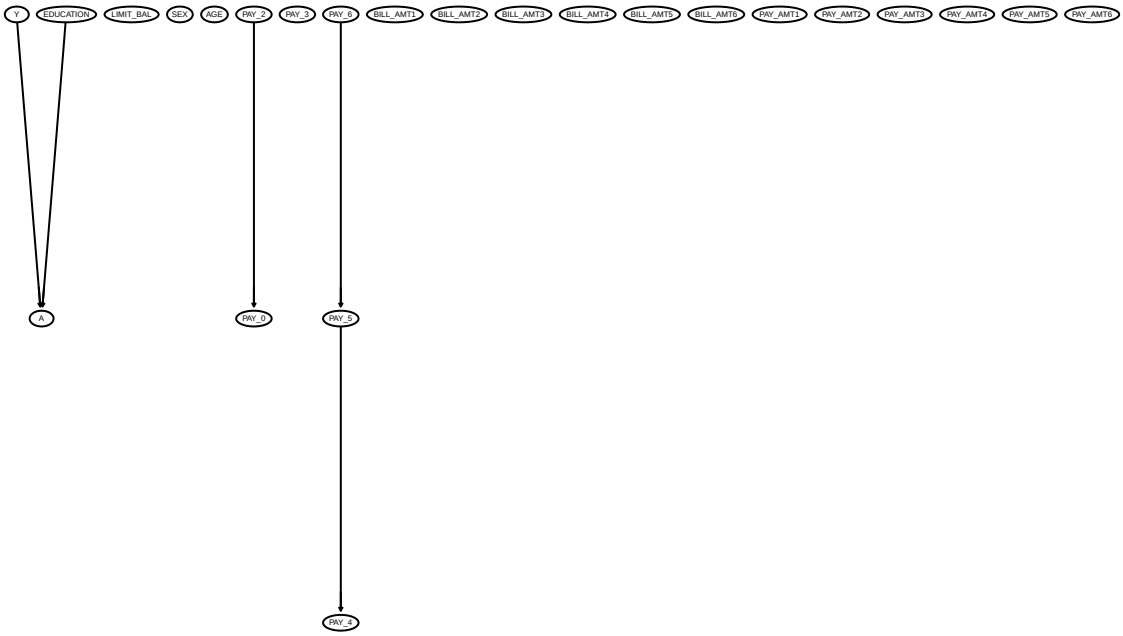
```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_cextend_maxsx_test.png", width = 
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```

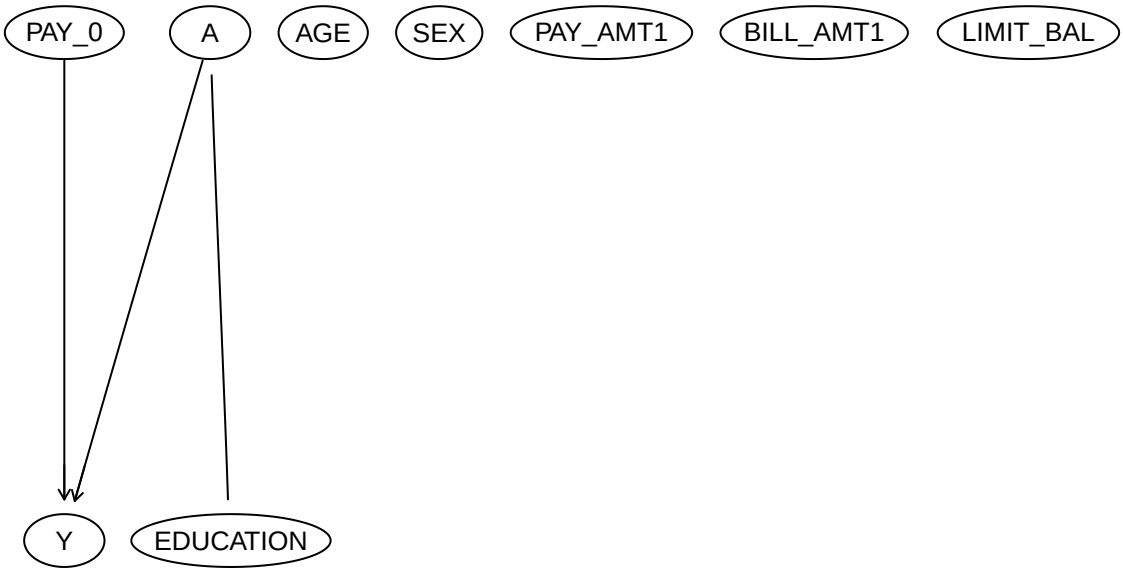



```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_cextend_maxsx_test.png", width = 
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

Since we see this isn't working well and the DAG is very sparse, let's try just limiting to the most recent variables, which should have stronger relationships with Y and thus be easier for the PC algorithm to find edges.

```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_limitedcov, shape = "ellipse",
+       main = "Causal DAG: PC Method, Limited Covariates")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_limitedcov.png", width = 1600, hei
> graphviz.plot(pc_fit_bnlearn_limitedcov, shape = "ellipse",
+       main = "Causal DAG: PC Method, Limited Covariates")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> # Find indices of variables connected to Y
> pc_y_indices <- bnlearn::nbr(pc_fit_bnlearn_limitedcov, "Y") #all neighbors - but not spouses
> print(pc_y_indices)
```

```
[1] "A"     "PAY_0"
```

```
> #parents, children, spouses using cextend version of the graph, which orients some edges and thus all
> pc_y_indices_parents <- bnlearn::parents(pc_fit_bnlearn_limitedcov, "Y")
> pc_y_indices_children <- bnlearn::children(pc_fit_bnlearn_limitedcov, "Y")
> pc_y_indices_spouses <- bnlearn::spouses(pc_fit_bnlearn_limitedcov, "Y")
> print("All relationships identified by PC")
```

```
[1] "All relationships identified by PC"
```

```
> print(pc_y_indices_children)
```

```
character(0)
```

```
> print(pc_y_indices_parents)
```

```
[1] "A"     "PAY_0"
```

```
> print(pc_y_indices_spouses)
```

```
character(0)
```

```
> pc_y_indices_all <- unique(c(pc_y_indices_parents, pc_y_indices_children, pc_y_indices_spouses, pc_y_
> print("All variables in Markov Blanket of Y identified by PC:")
```

```
[1] "All variables in Markov Blanket of Y identified by PC:"
```

```
> print(pc_y_indices_all)
```

```
[1] "A"     "PAY_0"
```

```
> # Update your covariate list for the models
> cov_pc <- setdiff(pc_y_indices_all, "A") #exclude A
> print(cov_pc) #only PAY_0?
```

```
[1] "PAY_0"
```

```
> pc_a_indices <- bnlearn::nbr(pc_fit_bnlearn_limitedcov, "A") #all neighbors - but not spouses
> print(pc_a_indices)
```

```
[1] "Y"         "EDUCATION"
```

```
> #parents, children, spouses using cextend version of the graph, which orients some edges and thus all
> pc_a_indices_parents <- bnlearn::parents(pc_fit_bnlearn_limitedcov, "A")
> pc_a_indices_children <- bnlearn::children(pc_fit_bnlearn_limitedcov, "A")
> pc_a_indices_spouses <- bnlearn::spouses(pc_fit_bnlearn_limitedcov, "A")
> print("All relationships identified by PC")
```

```
[1] "All relationships identified by PC"
```

```
> print(pc_a_indices_children)
```

```
[1] "Y"
```

```
> print(pc_a_indices_parents)
```

```
character(0)
```

```
> print(pc_a_indices_spouses)
```

```
[1] "PAY_0"
```

```
> pc_a_indices_all <- unique(c(pc_a_indices_parents, pc_a_indices_children, pc_a_indices_spouses, pc_a_
> print("All variables in Markov Blanket of A identified by PC:")
```

```
[1] "All variables in Markov Blanket of A identified by PC:"
```

```
> print(pc_a_indices_all)
```

```
[1] "Y"         "PAY_0"     "EDUCATION"
```

```
> cov_pc_a <-setdiff(pc_a_indices_all, "Y") #exclude Y, since we don't want to condition on the outcome
> print("Variables selected by PC (Markov Blanket of A):")
```

```
[1] "Variables selected by PC (Markov Blanket of A):"
```

```
> print(cov_pc_a)
```

```
[1] "PAY_0"     "EDUCATION"
```

19 pc algorithm nontransformed or

```
> #outcome regression
> if(length(cov_pc) == 0) {
+   formula_or_pc <- as.formula("Y ~ A")
+ } else {
+   formula_or_pc <- as.formula(paste("Y ~ A +", paste(cov_pc, collapse = "+")))
+ }
> model_or_pc <- glm(formula_or_pc, data = data, family = binomial)
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_or_pc)
```

```
Y ~ A + PAY_0
```

```
> # pc_data_all_enrolled <- data_all_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
> #   select(Y, A, all_of(cov_all))
>
> # pc_data_no_enrolled <- data_no_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
> #   select(Y, A, all_of(cov_all))
>
> # Predict default probability for world of everyone enrolled, and no one enrolled
> data$prob_Y_if_A1_pc_var<- predict(model_or_pc, newdata = data_all_enrolled, type = "response") #type
>
> EY_a1_or_pc_var <- mean(data$prob_Y_if_A1_pc_var)
>
> data$prob_Y_if_A0_pc_var <- predict(model_or_pc, newdata = data_no_enrolled, type = "response")
> EY_a0_or_pc_var <- mean(data$prob_Y_if_A0_pc_var)
>
> ace_or_pc_var <- EY_a1_or_pc_var - EY_a0_or_pc_var
>
> #print means
> print("Default likelihood (PC Set, OR), A=1 then A=0")
```

```
[1] "Default likelihood (PC Set, OR), A=1 then A=0"
```

```
> print(EY_a1_or_pc_var)
```

```
[1] 0.4284831
```

```
> print(EY_a0_or_pc_var)
```

```
[1] 0.2210747
```

```
> print(paste("OR ACE (PC Set):", round(ace_or_pc_var,4)))
```

```
[1] "OR ACE (PC Set): 0.2074"
```

20 PC algorithm nonttransformed ipw

```
> if(length(cov_pc_a) == 0) {
+   formula_ps_pc_var <- as.formula("A ~ 1")
+ } else {
+   formula_ps_pc_var <- as.formula(paste("A~", paste(cov_pc_a, collapse = "+"))) #formula of A~pc cova
+ }
> #get propensity scores
> model_ps_pc_var <- glm(formula_ps_pc_var, data = data, family = binomial) #binomial because only 2 ou
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_ps_pc_var)
```

```
A ~ PAY_0 + EDUCATION
```

```
> data$ps_pc_var <- predict(model_ps_pc_var, type = "response") #calculate propensity scores for all cu
> print("min and max ps")
```

```
[1] "min and max ps"
```

```
> print(min(data$ps_pc_var))
```

```
[1] 4.721032e-07
```

```
> print(max(data$ps_pc_var))
```

```
[1] 0.9999995
```

```
> #extreme ps means likely have extreme scores
>
> data$ps_pc_var_clamped <- pmax(pmin(data$ps_pc_var, 0.9999), 0.0001)
>
>
> # calculate weights.
> data$weights_pc_var <- ifelse(data[["A"]] == 1, 1 / data$ps_pc_var_clamped, 1 / (1 - data$ps_pc_var_cl
>
> print("min and max weights")
```

```
[1] "min and max weights"
```

```
> print(min(data$weights_pc_var)) #1.0001
```

```
[1] 1.0001
```

```
> print(max(data$weights_pc_var)) #2.8
```

```
[1] 7.066148
```

```
> #if customer was actually enrolled (A==1), then d weight as 1/ps, otherwise do 1/(1-ps)
> #So if customer had A=1 & ps=0.8, then weight = 1.25; but if A=0 then weight=5. Scales by unexpectedn
>
> # Estimate ACE using the weighted average
> #filter to only people who were enrolled A=1 or A=0
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
> # Filter to A0 or A1 groups, get default and multiply by weight; basically, if paid on time, contribu
> EY_a0_ipw_pc_var <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_pc_var)/sum(a0_group$w
> EY_a1_ipw_pc_var <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_pc_var)/sum(a1_group$w
```

```
>
> ace_ipw_pc_var <- EY_a1_ipw_pc_var - EY_a0_ipw_pc_var
> print(paste("IPW ACE (PC set):", round(ace_ipw_pc_var,4)))
```

```
[1] "IPW ACE (PC set): 0.2064"
```

21 Pc algorithm bootstrap nontransformed

```
> bootstrap_results_pc <- replicate(bootstrap_sample_size, {
+   #bootstrap sample
+   sample_df <- data[sample(nrow(data), replace = TRUE), ]
+
+   #convert data to numerical for gaussian tests
+   sample_df_pc_data <- sample_df %>%
+     mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+     select(Y, A, all_of(cov_all))
+
+   cov_most_recent <- c("PAY_0", "EDUCATION", "AGE", "SEX", "PAY_AMT1", "BILL_AMT1", "LIMIT_BAL")
+
+   sample_df_pc_data_limitedcov <- sample_df_pc_data %>%
+     select(Y, A, all_of(cov_most_recent))
+
+   pc_fit_boot <- pc.stable(sample_df_pc_data_limitedcov,
+                     # blacklist = master_blacklist,
+                       # test = "mix", #not needed, will automatically detect which test based on vari
+                     alpha = 0.01)
+
+   adj_matrix <- bnlearn::amat(pc_fit_boot)
+
+   #extract markov blanket for y
+   y_indices_boot <- bnlearn::nbr(pc_fit_boot, "Y") #all neighbors - but not spouses
+
+   #parents, children, spouses using cextend version of the graph, which orients some edges and thus a
+   y_indices_parents_boot <- bnlearn::parents(pc_fit_boot, "Y")
+   y_indices_children_boot <- bnlearn::children(pc_fit_boot, "Y")
+   y_indices_spouses_boot <- bnlearn::spouses(pc_fit_boot, "Y")
+
+   y_indices_all_boot <- unique(c(y_indices_parents_boot, y_indices_children_boot, y_indices_spouses_b
+
+   # Update  covariate list for the models
+   cov_pc_boot <- setdiff(y_indices_all_boot, "A")
+
+
+   #now for a
+   a_indices_boot <- bnlearn::nbr(pc_fit_boot, "A") #all neighbors - but not spouses
+   print(a_indices_boot)
+
+   #parents, children, spouses using cextend version of the graph, which orients some edges and thus a
+   a_indices_parents_boot <- bnlearn::parents(pc_fit_boot, "A")
+   a_indices_children_boot <- bnlearn::children(pc_fit_boot, "A")
+   a_indices_spouses_boot <- bnlearn::spouses(pc_fit_boot, "A")
+
+   a_indices_all_boot <- unique(c(a_indices_parents_boot, a_indices_children_boot, a_indices_spouses_b
```

```
+
+   cov_pc_boot_a <-setdiff(a_indices_all_boot, "Y")
+
+   # what if the blanket is empty? or
+   if(length(cov_pc_boot) == 0) {
+     formula_or_pc_boot <- as.formula("Y ~ A")
+   } else {
+     formula_or_pc_boot <- as.formula(paste("Y ~ A +", paste(cov_pc_boot, collapse = "+")))
+   }
+   #ipw
+   if(length(cov_pc_boot_a) == 0) {
+     formula_ps_pc_boot <- as.formula("A ~ 1")
+   } else {
+     formula_ps_pc_boot <- as.formula(paste("A ~", paste(cov_pc_boot_a, collapse = "+")))
+   }
+
+   # outcome regression
+   model_or_pc_boot <- glm(formula_or_pc_boot, data = sample_df, family = binomial)
+
+   #create fake datasets
+   sample_df_a0 <- sample_df
+   sample_df_a0$A <- factor(0, levels = levels(sample_df$A))
+   sample_df_a1 <- sample_df
+   sample_df_a1$A <- factor(1, levels = levels(sample_df$A))
+
+   # Predict default probability for world of everyone enrolled, and no one enrolled
+   sample_df_prob_Y_if_A1 <- predict(model_or_pc_boot, newdata = sample_df_a1, type = "response")
+   EY_a1_or_sample_df <- mean(sample_df_prob_Y_if_A1)
+
+   sample_df_prob_Y_if_A0 <- predict(model_or_pc_boot, newdata = sample_df_a0, type = "response")
+   EY_a0_or_sample_df <- mean(sample_df_prob_Y_if_A0)
+
+   ace_or_sample_df <- EY_a1_or_sample_df - EY_a0_or_sample_df
+
+   #IPW
+   #get propensity score
+   model_ps_sample_df <- glm(formula_ps_pc_boot, data = sample_df, family = binomial)
+   sample_df$ps_sample_df <- predict(model_ps_sample_df, type = "response")
+   #clamp ps score between 0.0001 and 0.9999 because otherwise get warnings of fitted prob being 0 or
+   sample_df$ps_sample_df_clamped <- pmax(pmin(sample_df$ps_sample_df, 0.9999), 0.0001)
+   #get weights
+   sample_df$weights_sample_df <- ifelse(sample_df$A == 1, 1/sample_df$ps_sample_df_clamped, 1/(1-sampl
+   #divide into a0 and a1 groups
+   a0_group_sample_df <- sample_df[sample_df$A == 0, ]
+   a1_group_sample_df <- sample_df[sample_df$A == 1, ]
+   #calculate ACE
+   EY_a0_ipw_sample_df <- sum(as.numeric(as.character(a0_group_sample_df$Y)) * a0_group_sample_df$weigh
+   EY_a1_ipw_sample_df <- sum(as.numeric(as.character(a1_group_sample_df$Y)) * a1_group_sample_df$weigh
+   ace_ipw_sample_df <-  EY_a1_ipw_sample_df - EY_a0_ipw_sample_df
+
+   return(list(ace_or = ace_or_sample_df,
+               ace_ipw = ace_ipw_sample_df,
+               adj_matrix = adj_matrix,
```

```
+                 blanket = y_indices_all_boot,
+                 blanket_a = a_indices_all_boot
+                 ))
+ }, simplify = FALSE)
```

```
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
```

```
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "Y"

[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "EDUCATION"

[1] "Y"          "EDUCATION"
character(0)

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
```

```
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
character(0)

[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"     "PAY_0"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
character(0)
[1] "Y"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"

[1] "Y"
character(0)

[1] "Y"   "SEX"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION" "SEX"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
character(0)

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"   "AGE"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"
[1] "Y"
[1] "Y"
```

```
character(0)
[1] "Y"
[1] "Y"
character(0)
[1] "Y"
[1] "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
```

```
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "PAY_0"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
```

```
[1] "Y"
[1] "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "EDUCATION"
character(0)
[1] "Y"
[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
character(0)
```

```
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
character(0)

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
character(0)
[1] "Y"

[1] "Y"
character(0)

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"   "AGE"

[1] "Y"
[1] "Y"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "EDUCATION"
[1] "Y"
character(0)

[1] "Y"
[1] "Y"
character(0)
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
```

```
[1] "Y"         "EDUCATION"

[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
character(0)

[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "EDUCATION" "SEX"

[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
```

```
[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"    "SEX"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"
[1] "Y"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"

[1] "Y"          "EDUCATION"
[1] "EDUCATION"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
```

```
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
character(0)
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
```

```
[1] "Y"         "EDUCATION"
character(0)
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION" "SEX"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
```

```
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"   "SEX"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION" "SEX"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
```

```
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
```

```
[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"   "SEX"

[1] "Y"         "EDUCATION"
[1] "Y"     "PAY_0"

[1] "Y"
character(0)

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"
character(0)
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
```

```
[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"   "SEX"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
```

```
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
character(0)
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"     "PAY_0"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
```

```
[1] "Y"
[1] "Y"
[1] "Y"     "PAY_0"
[1] "Y"
[1] "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
character(0)
[1] "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "EDUCATION"
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"
character(0)
[1] "Y"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
[1] "Y"         "EDUCATION"
character(0)
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"         "EDUCATION"
```

```
[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"
[1] "Y"

[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
character(0)
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "Y"
[1] "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "Y"

[1] "Y"         "EDUCATION"
[1] "EDUCATION"

[1] "Y"         "EDUCATION"

[1] "Y"         "EDUCATION"
```

```
> orig_ace_pc_var <- c(ace_or_pc_var, ace_ipw_pc_var)
>
> #extract ACE values
> boot_aces_or_pc  <- unlist(lapply(bootstrap_results_pc, function(x) x$ace_or))
> boot_aces_ipw_pc <- unlist(lapply(bootstrap_results_pc, function(x) x$ace_ipw))
```

```
>
> #extract markov blanket
> boot_blankets_pc <- lapply(bootstrap_results_pc, function(x) x$blanket)
> boot_blankets_pc_a <- lapply(bootstrap_results_pc, function(x) x$blanket_a)
>
> # convert to dataframe
> boot_df_pc_var <- data.frame(
+   Regression = boot_aces_or_pc,
+   IPW = boot_aces_ipw_pc
+ )
> colnames(boot_df_pc_var) <- c("Regression", "IPW")
>
> # Calculate Metrics
> metrics_pc_var <- data.frame(
+   Method = c("Regression", "IPW"),
+   Original_ACE = orig_ace_pc_var, # Use values from the full dataset
+   Bootstrap_ACE = c(mean(boot_df_pc_var$Regression), mean(boot_df_pc_var$IPW)),
+   Standard_Error = c(sd(boot_df_pc_var$Regression), sd(boot_df_pc_var$IPW))
+ )
>
> # Calculate Bias
> metrics_pc_var$Bootstrap_Bias <- metrics_pc_var$Bootstrap_ACE - metrics_pc_var$Original_ACE
> print(metrics_pc_var)

      Method Original_ACE Bootstrap_ACE Standard_Error Bootstrap_Bias
1 Regression    0.2074085     0.2082829     0.03996669   0.0008744523
2        IPW    0.2063976     0.2002647     0.04624014  -0.0061329008
> boot_blankets_pc_unlisted <- unlist(boot_blankets_pc)
>
> boot_mb_stability_pc <- table(boot_blankets_pc_unlisted) / bootstrap_sample_size
> print("Selection Frequency for Y's Markov Blanket:")

[1] "Selection Frequency for Y's Markov Blanket:"
> print(sort(boot_mb_stability_pc, decreasing = TRUE))

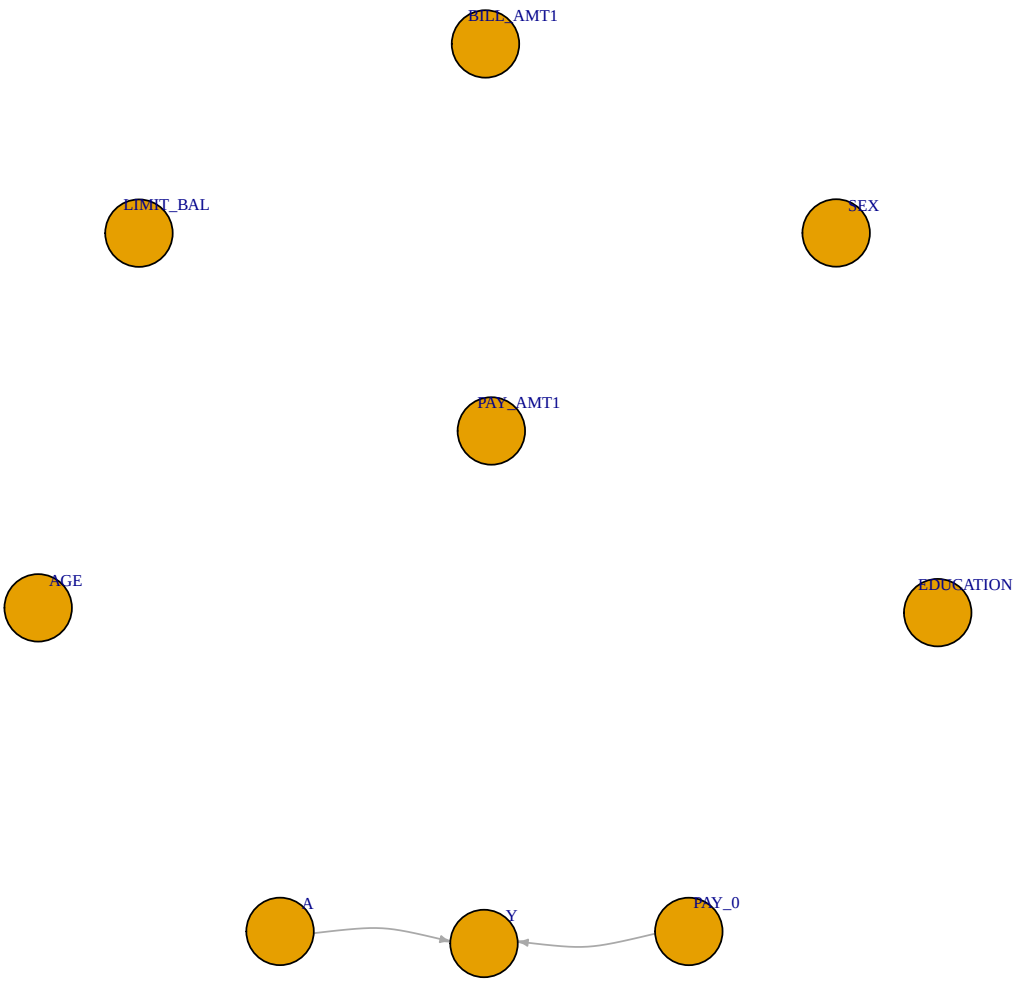
boot_blankets_pc_unlisted
    PAY_0         A EDUCATION       SEX
    0.992     0.849     0.174     0.034
> boot_blankets_pc_a_unlisted <- unlist(boot_blankets_pc_a)
>
> boot_mb_stability_pc_a <- table(boot_blankets_pc_a_unlisted) / bootstrap_sample_size
> print("Selection Frequency for A's Markov Blanket:")

[1] "Selection Frequency for A's Markov Blanket:"
> print(sort(boot_mb_stability_pc_a, decreasing = TRUE))

boot_blankets_pc_a_unlisted
        Y     PAY_0 EDUCATION       SEX       AGE
    0.849     0.668     0.473     0.032     0.002
> #get adjacent matrices
> all_adj_bootstrap_pc <- lapply(bootstrap_results_pc, function(x) x$adj_matrix)
> #sum them up to get edge frequencies - prob an edge exists between 2 variables
```

```
> consensus_matrix_bootstrap_pc <- Reduce("+", all_adj_bootstrap_pc) / bootstrap_sample_size #reduce su
>
> # Create a graph where an edge exists if it appeared in > 50% of samples
> average_adj_bootstrap_pc <- (consensus_matrix_bootstrap_pc > 0.5) * 1
>
> # Convert back to a graph object for plotting
> avg_graph_bootstrap_pc <- graph_from_adjacency_matrix(average_adj_bootstrap_pc, mode = "directed")
>
> plot(avg_graph_bootstrap_pc,
+      main = "Consensus DAG (Edges in >50% of Bootstraps): PC Method, Limited Covariates",
+      vertex.size = 15,              # Smaller nodes
+      vertex.label.cex = 0.6,        # Smaller labels
+      vertex.label.dist = 1.2,       # Offset labels from node centers
+      edge.arrow.size = 0.3,         # Smaller arrows
+      edge.curved = 0.2)             # Slight curve to distinguish overlapping arrows
```

Consensus DAG (Edges in >50% of Bootstraps): PC Method, Limited Covariates




```
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/consensus_dag_pc_bootstrap_limitedcov.png", width = 1200, heigh
> par(mar = c(1, 1, 3, 1))
> plot(avg_graph_bootstrap_pc,
```

```
+      main = "Consensus DAG (Edges in >50% of Bootstraps): PC Method, Limited Covariates",
+      vertex.size = 15,              # Smaller nodes
+      vertex.label.cex = 0.6,        # Smaller labels
+      vertex.label.dist = 1.2,       # Offset labels from node centers
+      edge.arrow.size = 0.3,         # Smaller arrows
+      edge.curved = 0.2)             # Slight curve to distinguish overlapping arrows
> dev.off()
```

```
pdf
  2
```

22 5 Now the Analysis, for PC-selected variables! (full cov)

```
> pc_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
> # pc_data_var_types <- ifelse(sapply(pc_data, is.factor), 1, 0)
> print(summary(pc_data)) #proves data is in right format.
```

```
 Y       A         LIMIT_BAL      SEX     EDUCATION      AGE
 0:344   0:279   Min.   : 10000   0:291   1:176     Min.   :21.00
 1:156   1:221   1st Qu.: 50000   1:209   2:241     1st Qu.:28.00
                 Median :140000           3: 74     Median :34.00
                 Mean   :164060           4:  2     Mean   :35.35
                 3rd Qu.:230000           5:  7     3rd Qu.:40.00
                 Max.   :620000                     Max.   :65.00

    PAY_0     PAY_2    PAY_3    PAY_4    PAY_5    PAY_6      BILL_AMT1
 0      :232   -2: 64   -2: 68   -2: 74   -2: 72   -2: 76   Min.   :  -532
 -1     :100   -1:109   -1:115   -1:106   -1: 99   -1:102   1st Qu.:  3744
 1      : 71   0 :255   0 :255   0 :248   0 :270   0 :265   Median : 22156
 -2     : 48   2 : 63   2 : 57   2 : 69   2 : 56   2 : 54   Mean   : 49072
 2      : 42   3 :  5   3 :  4   3 :  3   3 :  2   3 :  3   3rd Qu.: 58694
 3      :  4   4 :  3   4 :  1            4 :  1            Max.   :604019
 (Other):  3   5 :  1
   BILL_AMT2        BILL_AMT3        BILL_AMT4        BILL_AMT5
 Min.   :-17710   Min.   :-17706   Min.   : -2800   Min.   :-81334
 1st Qu.:  2418   1st Qu.:  2583   1st Qu.:  2888   1st Qu.:  2451
 Median : 19870   Median : 19507   Median : 18654   Median : 18538
 Mean   : 45531   Mean   : 42682   Mean   : 40983   Mean   : 38318
 3rd Qu.: 55343   3rd Qu.: 51370   3rd Qu.: 49070   3rd Qu.: 45385
 Max.   :605943   Max.   :467911   Max.   :404157   Max.   :587067

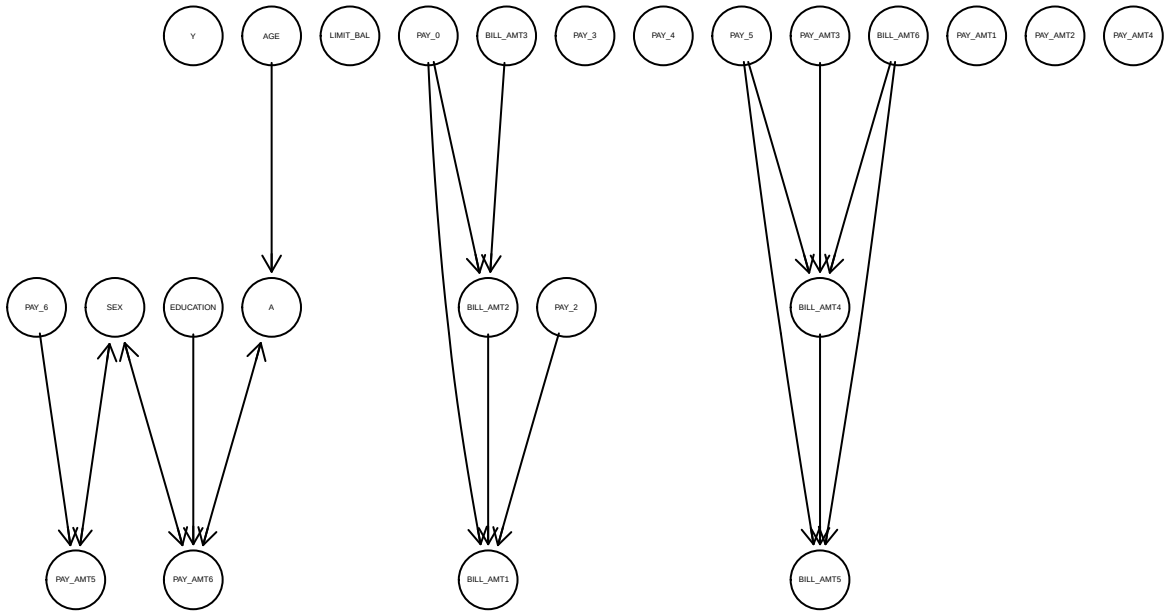
   BILL_AMT6         PAY_AMT1         PAY_AMT2         PAY_AMT3
 Min.   : -2506   Min.   :     0   Min.   :     0   Min.   :     0.0
 1st Qu.:  1228   1st Qu.:  1000   1st Qu.:  1000   1st Qu.:   498.2
 Median : 17972   Median :  2000   Median :  2038   Median :  1737.5
 Mean   : 36065   Mean   :  4940   Mean   :  5766   Mean   :  5872.4
 3rd Qu.: 44432   3rd Qu.:  4753   3rd Qu.:  5000   3rd Qu.:  4906.8
 Max.   :323821   Max.   :234000   Max.   :294318   Max.   :319494.0

    PAY_AMT4         PAY_AMT5         PAY_AMT6
 Min.   :     0   Min.   :     0   Min.   :     0.0
```

```
 1st Qu.:   298   1st Qu.:   390   1st Qu.:   181.5
 Median :  1576   Median :  1529   Median :  1500.0
 Mean   :  4258   Mean   :  5212   Mean   :  5084.9
 3rd Qu.:  3956   3rd Qu.:  4000   3rd Qu.:  4134.0
 Max.   :109332   Max.   :331788   Max.   :189600.0
```

```
> # attr(pc_data, "adaptDF") <- TRUE
>
> # Run PC; alpha serves as a tuning parameter for the significance of independence tests
> pc_fit_test_full_fin <- pc(suffStat = pc_data,
+                     indepTest = mixCItest,
+                     labels = colnames(pc_data),
+                     alpha = 0.01,
+                     verbose = FALSE,
+                     u2pd = "relaxed",
+                     skel.method = "stable" ,  #order of test doesnt matter
+                     maj.rule = FALSE,
+                     solve.confl = TRUE #if some say a-->b and some say b-->a, then pick a direction
+                     )
>
> # Plot DAG
> plot(pc_fit_test_full_fin@graph, main = "Causal DAG: PC Method, All Covariates")
```

Causal DAG: PC Method, All Covariates




```
> #export
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_fullcov.png", width = 1200, height = 1000
>
> plot(pc_fit_test_full_fin@graph, main = "Causal DAG: PC Method, All Covariates")
>
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> pc_full_dag <- pcalg2dagitty(as(pc_fit_test_full_fin@graph, "matrix"),
+                         labels = colnames(pc_data),
+                         type = "cpdag")
>
> # Get the Markov Blanket (Parents + Children + Spouses)
> pc_full_mb_y <- markovBlanket(pc_full_dag, "Y")
> print(pc_full_mb_y)
```

```
character(0)
```

```
> # Update your covariate list for the models
> cov_pc_full <- setdiff(pc_full_mb_y, "A") #exclude A
> print(cov_pc_full) #only PAY_0?
```

```
character(0)
```

```
> # Get the Markov Blanket (Parents + Children + Spouses)
> pc_full_mb_a <- markovBlanket(pc_full_dag, "A")
> print(pc_full_mb_a)
```

```
[1] "AGE"
```

```
> # Update your covariate list for the models
> cov_pc_full_a <- setdiff(pc_full_mb_a, "Y") #exclude A
> print(cov_pc_full_a) #only PAY_0?
```

```
[1] "AGE"
```

23 pc algorithm nontransformed or

```
> #outcome regression
> if(length(cov_pc_full) == 0) {
+   formula_or_pc_full <- as.formula("Y ~ A")
+ } else {
+   formula_or_pc_full <- as.formula(paste("Y ~ A +", paste(cov_pc_full, collapse = "+")))
+ }
> model_or_pc_full <- glm(formula_or_pc_full, data = data, family = binomial)
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_or_pc_full)
```

```
Y ~ A
```

```
> # pc_data_all_enrolled <- data_all_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
> #   select(Y, A, all_of(cov_all))
>
> # pc_data_no_enrolled <- data_no_enrolled %>%
> #   # mutate(across(everything(), as.numeric)) %>%
> #   select(Y, A, all_of(cov_all))
>
```

```
> # Predict default probability for world of everyone enrolled, and no one enrolled
> data$prob_Y_if_A1_pc_var_full<- predict(model_or_pc_full, newdata = data_all_enrolled, type = "respons
>
> EY_a1_or_pc_var_full <- mean(data$prob_Y_if_A1_pc_var_full)
>
> data$prob_Y_if_A0_pc_var_full <- predict(model_or_pc_full, newdata = data_no_enrolled, type = "respons
> EY_a0_or_pc_var_full <- mean(data$prob_Y_if_A0_pc_var_full)
>
> ace_or_pc_var_full <- EY_a1_or_pc_var_full - EY_a0_or_pc_var_full
>
> #print means
> print("Default likelihood (PC Set, OR), A=1 then A=0")
```

```
[1] "Default likelihood (PC Set, OR), A=1 then A=0"
```

```
> print(EY_a1_or_pc_var_full)
```

```
[1] 0.4162896
```

```
> print(EY_a0_or_pc_var_full)
```

```
[1] 0.2293907
```

```
> print(paste("OR ACE (PC Set):", round(ace_or_pc_var_full,4)))
```

```
[1] "OR ACE (PC Set): 0.1869"
```

24 PC algorithm nonttransformed ipw

```
> if(length(cov_pc_full_a) == 0) {
+   formula_ps_pc_var_full <- as.formula("A ~ 1")
+ } else {
+   formula_ps_pc_var_full <- as.formula(paste("A~", paste(cov_pc_full_a, collapse = "+"))) #formula of
+ }
> #get propensity scores
> model_ps_pc_var_full <- glm(formula_ps_pc_var_full, data = data, family = binomial) #binomial because
>
> print("formula")
```

```
[1] "formula"
```

```
> print(formula_ps_pc_var_full)
```

```
A ~ AGE
```

```
> data$ps_pc_var_full <- predict(model_ps_pc_var_full, type = "response") #calculate propensity scores
> print("min and max ps")
```

```
[1] "min and max ps"
```

```
> print(min(data$ps_pc_var_full))
```

```
[1] 0.1174343
```

```
> print(max(data$ps_pc_var_full))
```

```
[1] 0.9681131
```

```
> #extreme ps means likely have extreme scores
>
> data$ps_pc_var_clamped_full <- pmax(pmin(data$ps_pc_var_full, 0.9999), 0.0001)
>
>
> # calculate weights.
> data$weights_pc_var_full <- ifelse(data[["A"]] == 1, 1 / data$ps_pc_var_clamped_full, 1 / (1 - data$ps
>
> print("min and max weights")
```

```
[1] "min and max weights"
```

```
> print(min(data$weights_pc_var_full)) #1.0001
```

```
[1] 1.032937
```

```
> print(max(data$weights_pc_var_full)) #2.8
```

```
[1] 9.837797
```

```
> #if customer was actually enrolled (A==1), then d weight as 1/ps, otherwise do 1/(1-ps)
> #So if customer had A=1 & ps=0.8, then weight = 1.25; but if A=0 then weight=5. Scales by unexpectedn
>
> # Estimate ACE using the weighted average
> #filter to only people who were enrolled A=1 or A=0
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
> # Filter to A0 or A1 groups, get default and multiply by weight; basically, if paid on time, contribu
> EY_a0_ipw_pc_var_full <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_pc_var_full)/sum
> EY_a1_ipw_pc_var_full <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_pc_var_full)/sum
>
> ace_ipw_pc_var_full <- EY_a1_ipw_pc_var_full - EY_a0_ipw_pc_var_full
> print(paste("IPW ACE (PC set):", round(ace_ipw_pc_var_full,4)))
```

```
[1] "IPW ACE (PC set): 0.1778"
```

25 6 Now the Analysis, for Local discovery methods

Use MMPC from bnlearn. The max-min parents and children algorithm. This searches the neighborhood of Y (parents and children) by testing for conditional independence at the alpha=0.01 level. Variable is only kept if relationship with Y can't be explained away by other variables.

26 nontransformed, local method

IAMB (Incremental Association)

The most common "Gold Standard" for medium-sized datasets.

```
How it works: It is dynamic. In every step of the growth phase, it re-tests all remaining variables and

The Pros: Much more reliable at finding the correct parents, children, and spouses than GS.

The Cons: Slower than GS (it runs O(n2) tests).

N=500 Performance: Recommended. This is the most balanced choice for your credit data.
```

27 Test IAMB

```
> #convert data to numerical
> local_data_test <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
>
> local_data_test_mbonly <- local_data_test %>%
+     mutate(across(everything(), function(x) {
+       if(is.factor(x) || is.character(x)) as.numeric(as.factor(x)) else rank(x)
+     }))
>
> # Run nbr to find the markov blanket of Y only
> local_y_test <- learn.mb(local_data_test_mbonly, node = "Y", method = "iamb", alpha = 0.01, test="cor
>
> print("Variables in the Local Neighborhood (Parents/Children) of Y:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of Y:"
```

```
> print(local_y_test)
```

```
[1] "PAY_0" "A"
```

```
> # Extract adjustment set (exclude treatment A)
> cov_local_test <- setdiff(local_y_test, "A") #L
> print(cov_local_test)
```

```
[1] "PAY_0"
```

```
> #now do the same for A to get covariates for propensity score model
> local_a_test <- learn.mb(local_data_test_mbonly, node = "A", method = "iamb", alpha = 0.01, test="cor
> print("Variables in the Local Neighborhood (Parents/Children) of A:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of A:"
```

```
> print(local_a_test)
```

```
[1] "AGE"       "Y"         "EDUCATION"
```

```
> cov_local_a_test <-setdiff(local_a_test, "Y") #exclude Y, since we don't want to condition on the out
> print(cov_local_a_test)
```

```
[1] "AGE"       "EDUCATION"
```

```
> # learn.mb(local_data, node = "A", method = "a", alpha = 0.01)
> #Error in check.label(algorithm, choices = ok, labels = learning.labels,  :  valid learning algorithm
```

```
> #convert data to numerical
> local_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
>
> local_data_test_mbonly <- local_data %>%
+     mutate(across(everything(), function(x) {
+       if(is.factor(x) || is.character(x)) as.numeric(as.factor(x)) else rank(x)
+     }))
>
>
```

```
> # Run nbr to find the markov blanket of Y only
> local_y <- learn.mb(local_data_test_mbonly, node = "Y", method = "inter.iamb", alpha = 0.01, test = "
>
> print("Variables in the Local Neighborhood (Parents/Children) of Y:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of Y:"
```

```
> print(local_y)
```

```
[1] "PAY_0" "A"
```

```
> # Extract adjustment set (exclude treatment A)
> cov_local <- setdiff(local_y, "A") #L
> print(cov_local)
```

```
[1] "PAY_0"
```

```
> #now do the same for A to get covariates for propensity score model
> local_a <- learn.mb(local_data_test_mbonly, node = "A", method = "inter.iamb", alpha = 0.01, test = "
> print("Variables in the Local Neighborhood (Parents/Children) of A:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of A:"
```

```
> print(local_a)
```

```
[1] "AGE"       "Y"         "EDUCATION"
```

```
> cov_local_a <-setdiff(local_a, "Y") #exclude Y, since we don't want to condition on the outcome in th
> print(cov_local_a)
```

```
[1] "AGE"       "EDUCATION"
```

28 local algorithm nontransformed or

```
> # --- Outcome Regression (Local Set) ---
> if (length(cov_local) == 0) {
+   formula_or_local <- as.formula("Y ~ A")
+ } else {
+   formula_or_local <- as.formula(paste("Y ~ A +", paste(cov_local, collapse = " + ")))
+ }
>
> model_or_local <- glm(formula_or_local, data = data, family = binomial)
>
> # Counterfactual predictions
> EY_a1_or_local <- mean(predict(model_or_local, newdata = data_all_enrolled, type = "response"))
> EY_a0_or_local <- mean(predict(model_or_local, newdata = data_no_enrolled, type = "response"))
> ace_or_local <- EY_a1_or_local - EY_a0_or_local
>
> print(paste("Local Discovery OR ACE:", round(ace_or_local, 4)))
```

```
[1] "Local Discovery OR ACE: 0.2074"
```

29 local algorithm nontransformed ipw

```
> if (length(cov_local_a) == 0) {
+   formula_ps_local <- as.formula("A ~ 1")
```

```
+ } else {
+   formula_ps_local <- as.formula(paste("A ~", paste(cov_local_a, collapse = " + ")))
+ }
>
> model_ps_local <- glm(formula_ps_local, data = data, family = binomial)
> data$ps_local <- predict(model_ps_local, type = "response")
> data$ps_local_clamped <- pmax(pmin(data$ps_local, 0.9999), 0.0001)
>
> # Weights
> data$weights_local <- ifelse(data$A == 1, 1 / data$ps_local_clamped, 1 / (1 - data$ps_local_clamped))
>
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
>
>
> EY_a0_ipw_local <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_local)/sum(a0_group$wei
> EY_a1_ipw_local <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_local)/sum(a1_group$wei
>
> ace_ipw_local <- EY_a1_ipw_local - EY_a0_ipw_local
> print(paste("Local Discovery IPW ACE:", round(ace_ipw_local, 4)))
```

```
[1] "Local Discovery IPW ACE: 0.1791"
```

30 local algorithm nontransformed bootstrap

```
> bootstrap_results_local <- replicate(bootstrap_sample_size, {
+   sample_df <- data[sample(nrow(data), replace = TRUE), ]
+
+
+   sample_df_local <- sample_df %>%
+     mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+     select(Y, A, all_of(cov_all))
+
+   # This maps factors to integers and continuous variables to ranks
+   sample_df_ranked <- sample_df_local %>%
+     mutate(across(everything(), function(x) {
+       if(is.factor(x) || is.character(x)) as.numeric(as.factor(x)) else rank(x)
+     }))
+
+   # We use sample_df_ranked, method = "inter.iamb", and test = "cor"
+   local_y_boot <- tryCatch({
+     learn.mb(sample_df_ranked, node = "Y", method = "inter.iamb", test = "cor", alpha = 0.01)
+   }, error = function(e) character(0))
+
+   cov_local_boot <- setdiff(local_y_boot, "A")
+
+   local_a_boot <- tryCatch({
+     learn.mb(sample_df_ranked, node = "A", method = "inter.iamb", test = "cor", alpha = 0.01)
+   }, error = function(e) character(0))
+
+   cov_local_boot_a <- setdiff(local_a_boot, "Y")
+
```

```
+
+   if (length(cov_local_boot) == 0) {
+     formula_or_local_boot <- as.formula("Y ~ A")
+   } else {
+     formula_or_local_boot <- as.formula(paste("Y ~ A +", paste(cov_local_boot, collapse = " + ")))
+   }
+   if (length(cov_local_boot_a) == 0) {
+     formula_ps_local_boot <- as.formula("A ~ 1")
+   } else {
+     formula_ps_local_boot <- as.formula(paste("A ~", paste(cov_local_boot_a, collapse = " + ")))
+   }
+
+
+   # Uses sample_df so the GLM correctly processes the original factors/scales
+   model_or_local_boot <- glm(formula_or_local_boot, data = sample_df, family = binomial)
+
+   # create fake datasets
+   sample_df_a0 <- sample_df
+   sample_df_a0$A <- factor(0, levels = levels(sample_df$A))
+   sample_df_a1 <- sample_df
+   sample_df_a1$A <- factor(1, levels = levels(sample_df$A))
+
+   sample_df_prob_Y_if_A1 <- predict(model_or_local_boot, newdata = sample_df_a1, type = "response")
+   EY_a1_or_sample_df <- mean(sample_df_prob_Y_if_A1)
+
+   sample_df_prob_Y_if_A0 <- predict(model_or_local_boot, newdata = sample_df_a0, type = "response")
+   EY_a0_or_sample_df <- mean(sample_df_prob_Y_if_A0)
+
+   ace_or_sample_df <- EY_a1_or_sample_df - EY_a0_or_sample_df
+
+   # IPW
+   model_ps_local_boot <- glm(formula_ps_local_boot, data = sample_df, family = binomial)
+   sample_df$ps_sample_df <- predict(model_ps_local_boot, type = "response")
+   sample_df$ps_sample_df_clamped <- pmax(pmin(sample_df$ps_sample_df, 0.9999), 0.0001)
+   sample_df$weights_sample_df <- ifelse(sample_df$A == 1, 1/sample_df$ps_sample_df_clamped, 1/(1-sampl
+
+   a0_group_sample_df <- sample_df[sample_df$A== 0, ]
+   a1_group_sample_df <- sample_df[sample_df$A== 1, ]
+
+   EY_a0_ipw_sample_df <- sum(as.numeric(as.character(a0_group_sample_df$Y)) * a0_group_sample_df$weigh
+   EY_a1_ipw_sample_df <- sum(as.numeric(as.character(a1_group_sample_df$Y)) * a1_group_sample_df$weigh
+
+   ace_ipw_sample_df <- EY_a1_ipw_sample_df - EY_a0_ipw_sample_df
+
+   return(list(ace_or = ace_or_sample_df,
+         ace_ipw = ace_ipw_sample_df,
+         blanket = local_y_boot,
+         blanket_a = local_a_boot
+         ))
+ }, simplify=FALSE)
```

```
> orig_ace_local_var <- c(ace_or_local, ace_ipw_local)
>
> #extract ACE values
```

```
> boot_aces_or_local  <- unlist(lapply(bootstrap_results_local, function(x) x$ace_or))
> boot_aces_ipw_local <- unlist(lapply(bootstrap_results_local, function(x) x$ace_ipw))
>
> #extract markov blanket
> boot_blankets_local <- lapply(bootstrap_results_local, function(x) x$blanket)
> boot_blankets_local_a <- lapply(bootstrap_results_local, function(x) x$blanket_a)
>
>
> # convert to dataframe
> boot_df_local_var <- data.frame(
+   Regression = boot_aces_or_local,
+   IPW = boot_aces_ipw_local
+ )
> colnames(boot_df_local_var) <- c("Regression", "IPW")
>
> # Calculate Metrics
> metrics_local_var <- data.frame(
+   Method = c("Regression", "IPW"),
+   Original_ACE = orig_ace_local_var,
+   Bootstrap_ACE = c(mean(boot_df_local_var$Regression), mean(boot_df_local_var$IPW)),
+   Standard_Error = c(sd(boot_df_local_var$Regression), sd(boot_df_local_var$IPW))
+ )
>
> # Calculate Bias
> metrics_local_var$Bootstrap_Bias <- metrics_local_var$Bootstrap_ACE - metrics_local_var$Original_ACE
> print(metrics_local_var)
```

```
      Method Original_ACE Bootstrap_ACE Standard_Error Bootstrap_Bias
1 Regression    0.2074085     0.2072292     0.04103024   -0.000179315
2        IPW    0.1791043     0.1925573     0.04793317    0.013452935
```

```
> boot_blankets_local_unlisted <- unlist(boot_blankets_local)
>
> boot_mb_stability_local <- table(boot_blankets_local_unlisted) / bootstrap_sample_size
> print("Selection Frequency for Y's Markov Blanket:")
```

```
[1] "Selection Frequency for Y's Markov Blanket:"
```

```
> print(sort(boot_mb_stability_local, decreasing = TRUE))
```

```
boot_blankets_local_unlisted
        A     PAY_0     PAY_3  PAY_AMT2  PAY_AMT4 BILL_AMT1  PAY_AMT3     PAY_2
    0.987     0.950     0.377     0.177     0.177     0.158     0.141     0.114
EDUCATION LIMIT_BAL  PAY_AMT1       AGE     PAY_4     PAY_5  PAY_AMT5 BILL_AMT6
    0.107     0.061     0.057     0.051     0.029     0.021     0.020     0.019
BILL_AMT5 BILL_AMT2       SEX     PAY_6 BILL_AMT3 BILL_AMT4  PAY_AMT6
    0.018     0.013     0.013     0.012     0.011     0.008     0.007
```

```
> boot_blankets_local_unlisted_a <- unlist(boot_blankets_local_a)
>
> boot_mb_stability_local_a <- table(boot_blankets_local_unlisted_a) / bootstrap_sample_size
> print("Selection Frequency for A's Markov Blanket:")
```

```
[1] "Selection Frequency for A's Markov Blanket:"
```

```
> print(sort(boot_mb_stability_local_a, decreasing = TRUE))
```

```
boot_blankets_local_unlisted_a
      AGE         Y EDUCATION       SEX     PAY_0 LIMIT_BAL  PAY_AMT4  PAY_AMT6
    1.000     0.936     0.635     0.238     0.180     0.122     0.122     0.073
 PAY_AMT3     PAY_4 BILL_AMT2     PAY_2 BILL_AMT1 BILL_AMT5  PAY_AMT2 BILL_AMT6
    0.056     0.036     0.030     0.029     0.025     0.025     0.025     0.022
    PAY_3  PAY_AMT5     PAY_5 BILL_AMT3     PAY_6 BILL_AMT4  PAY_AMT1
    0.021     0.017     0.016     0.008     0.008     0.007     0.007
```

31 7 Now for the iamb with blacklist

```
> allowed_nodes_local_forbidden_edges <- c("Y", "A", cov_all)
> # Nothing can cause Demographics
> bl_demog <- expand.grid(from = allowed_nodes_local_forbidden_edges, to = c("AGE", "SEX", "EDUCATION"))
>
> # Y cannot cause anything
> bl_y <- expand.grid(from = "Y", to = setdiff(allowed_nodes_local_forbidden_edges, "Y"))
>
> # A cannot cause anything except Y
> bl_a <- expand.grid(from = "A", to = setdiff(allowed_nodes_local_forbidden_edges, c("A", "Y")))
>
> # temporal relationships: later months cannot cause earlier months
> pay_vars <- c("PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6")
> bill_amt_vars <- c("BILL_AMT1", "BILL_AMT2", "BILL_AMT3", "BILL_AMT4", "BILL_AMT5", "BILL_AMT6")
> pay_amt_vars <- c("PAY_AMT1", "PAY_AMT2", "PAY_AMT3", "PAY_AMT4", "PAY_AMT5", "PAY_AMT6")
>
> bl_temporal <- data.frame()
> for(i in 1:(length(pay_vars)-1)) {
+   for(j in (i+1):length(pay_vars)) {
+     # Block Present -> Past
+     bl_temporal <- rbind(bl_temporal, data.frame(from = pay_vars[i], to = pay_vars[j]))
+     bl_temporal <- rbind(bl_temporal, data.frame(from = bill_amt_vars[i], to = bill_amt_vars[j]))
+     bl_temporal <- rbind(bl_temporal, data.frame(from = pay_amt_vars[i], to = pay_amt_vars[j]))
+   }
+ }
>
> # Combine all into one master blacklist
> master_blacklist <- rbind(bl_demog, bl_y, bl_a, bl_temporal) %>%
+   mutate(across(everything(), as.character)) %>% # Convert everything to character to avoid factor le
+   filter(from != to) %>% # Remove self-loops
+   distinct()             # Remove duplicates
>
> print("Master Blacklist:")
```

```
[1] "Master Blacklist:"
```

```
> print(master_blacklist)
```

```
         from        to
1           Y       AGE
2           A       AGE
3   LIMIT_BAL       AGE
4         SEX       AGE
```

```
5   EDUCATION       AGE
6       PAY_0       AGE
7       PAY_2       AGE
8       PAY_3       AGE
9       PAY_4       AGE
10      PAY_5       AGE
11      PAY_6       AGE
12  BILL_AMT1       AGE
13  BILL_AMT2       AGE
14  BILL_AMT3       AGE
15  BILL_AMT4       AGE
16  BILL_AMT5       AGE
17  BILL_AMT6       AGE
18   PAY_AMT1       AGE
19   PAY_AMT2       AGE
20   PAY_AMT3       AGE
21   PAY_AMT4       AGE
22   PAY_AMT5       AGE
23   PAY_AMT6       AGE
24          Y       SEX
25          A       SEX
26  LIMIT_BAL       SEX
27  EDUCATION       SEX
28        AGE       SEX
29      PAY_0       SEX
30      PAY_2       SEX
31      PAY_3       SEX
32      PAY_4       SEX
33      PAY_5       SEX
34      PAY_6       SEX
35  BILL_AMT1       SEX
36  BILL_AMT2       SEX
37  BILL_AMT3       SEX
38  BILL_AMT4       SEX
39  BILL_AMT5       SEX
40  BILL_AMT6       SEX
41   PAY_AMT1       SEX
42   PAY_AMT2       SEX
43   PAY_AMT3       SEX
44   PAY_AMT4       SEX
45   PAY_AMT5       SEX
46   PAY_AMT6       SEX
47          Y EDUCATION
48          A EDUCATION
49  LIMIT_BAL EDUCATION
50        SEX EDUCATION
51        AGE EDUCATION
52      PAY_0 EDUCATION
53      PAY_2 EDUCATION
54      PAY_3 EDUCATION
55      PAY_4 EDUCATION
56      PAY_5 EDUCATION
57      PAY_6 EDUCATION
58  BILL_AMT1 EDUCATION
```

```
59  BILL_AMT2 EDUCATION
60  BILL_AMT3 EDUCATION
61  BILL_AMT4 EDUCATION
62  BILL_AMT5 EDUCATION
63  BILL_AMT6 EDUCATION
64   PAY_AMT1 EDUCATION
65   PAY_AMT2 EDUCATION
66   PAY_AMT3 EDUCATION
67   PAY_AMT4 EDUCATION
68   PAY_AMT5 EDUCATION
69   PAY_AMT6 EDUCATION
70          Y         A
71          Y LIMIT_BAL
72          Y     PAY_0
73          Y     PAY_2
74          Y     PAY_3
75          Y     PAY_4
76          Y     PAY_5
77          Y     PAY_6
78          Y BILL_AMT1
79          Y BILL_AMT2
80          Y BILL_AMT3
81          Y BILL_AMT4
82          Y BILL_AMT5
83          Y BILL_AMT6
84          Y  PAY_AMT1
85          Y  PAY_AMT2
86          Y  PAY_AMT3
87          Y  PAY_AMT4
88          Y  PAY_AMT5
89          Y  PAY_AMT6
90          A LIMIT_BAL
91          A     PAY_0
92          A     PAY_2
93          A     PAY_3
94          A     PAY_4
95          A     PAY_5
96          A     PAY_6
97          A BILL_AMT1
98          A BILL_AMT2
99          A BILL_AMT3
100         A BILL_AMT4
101         A BILL_AMT5
102         A BILL_AMT6
103         A  PAY_AMT1
104         A  PAY_AMT2
105         A  PAY_AMT3
106         A  PAY_AMT4
107         A  PAY_AMT5
108         A  PAY_AMT6
109    PAY_0     PAY_2
110 BILL_AMT1 BILL_AMT2
111  PAY_AMT1  PAY_AMT2
112    PAY_0     PAY_3
```

```
113 BILL_AMT1 BILL_AMT3
114  PAY_AMT1  PAY_AMT3
115     PAY_0     PAY_4
116 BILL_AMT1 BILL_AMT4
117  PAY_AMT1  PAY_AMT4
118     PAY_0     PAY_5
119 BILL_AMT1 BILL_AMT5
120  PAY_AMT1  PAY_AMT5
121     PAY_0     PAY_6
122 BILL_AMT1 BILL_AMT6
123  PAY_AMT1  PAY_AMT6
124     PAY_2     PAY_3
125 BILL_AMT2 BILL_AMT3
126  PAY_AMT2  PAY_AMT3
127     PAY_2     PAY_4
128 BILL_AMT2 BILL_AMT4
129  PAY_AMT2  PAY_AMT4
130     PAY_2     PAY_5
131 BILL_AMT2 BILL_AMT5
132  PAY_AMT2  PAY_AMT5
133     PAY_2     PAY_6
134 BILL_AMT2 BILL_AMT6
135  PAY_AMT2  PAY_AMT6
136     PAY_3     PAY_4
137 BILL_AMT3 BILL_AMT4
138  PAY_AMT3  PAY_AMT4
139     PAY_3     PAY_5
140 BILL_AMT3 BILL_AMT5
141  PAY_AMT3  PAY_AMT5
142     PAY_3     PAY_6
143 BILL_AMT3 BILL_AMT6
144  PAY_AMT3  PAY_AMT6
145     PAY_4     PAY_5
146 BILL_AMT4 BILL_AMT5
147  PAY_AMT4  PAY_AMT5
148     PAY_4     PAY_6
149 BILL_AMT4 BILL_AMT6
150  PAY_AMT4  PAY_AMT6
151     PAY_5     PAY_6
152 BILL_AMT5 BILL_AMT6
153  PAY_AMT5  PAY_AMT6
```

32 nontransformed, local method

IAMB (Incremental Association)

The most common “Gold Standard” for medium-sized datasets.

```
How it works: It is dynamic. In every step of the growth phase, it re-tests all remaining variables and

The Pros: Much more reliable at finding the correct parents, children, and spouses than GS.

The Cons: Slower than GS (it runs O(n2) tests).
```

N=500 Performance: Recommended. This is the most balanced choice for your credit data.

```
> #convert data to numerical
> local_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
>
> #because local discovery, need to exclude Y from blacklist...
> master_blacklist_excl_y <- master_blacklist %>%
+   filter(from != "Y" & to != "Y") %>%
+   as.matrix()
>
> master_blacklist_matrix_excl_y <- as.matrix(master_blacklist_excl_y)
>
> # Run nbr to find the markov blanket of Y only
> local_y_blacklist <- learn.mb(local_data, node = "Y", method = "inter.iamb", blacklist = master_blackl
>
> print("Variables in the Local Neighborhood (Parents/Children) of Y:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of Y:"
```

```
> print(local_y_blacklist)
```

```
character(0)
```

```
> # Extract adjustment set (exclude treatment A)
> cov_local_blacklist <- setdiff(local_y_blacklist, "A") #L
> print(cov_local_blacklist)
```

```
character(0)
```

```
> master_blacklist_excl_a <- master_blacklist %>%
+   filter(from != "A" & to != "A") %>%
+   as.matrix()
>
> master_blacklist_matrix_excl_a <- as.matrix(master_blacklist_excl_a)
>
>
> #now do the same for A to get covariates for propensity score model
> local_a_blacklist <- learn.mb(local_data, node = "A", method = "inter.iamb", blacklist = master_blackl
> print("Variables in the Local Neighborhood (Parents/Children) of A:")
```

```
[1] "Variables in the Local Neighborhood (Parents/Children) of A:"
```

```
> print(local_a_blacklist)
```

```
character(0)
```

```
> cov_local_a_blacklist <-setdiff(local_a_blacklist, "Y") #exclude Y, since we don't want to condition
> print(cov_local_a_blacklist)
```

```
character(0)
```

33 local algorithm nontransformed or

```
> # --- Outcome Regression (Local Set) ---
> if (length(cov_local_blacklist) == 0) {
```

```
+   formula_or_local_blacklist <- as.formula("Y ~ A")
+ } else {
+   formula_or_local_blacklist <- as.formula(paste("Y ~ A +", paste(cov_local_blacklist, collapse = " +
+ }
>
> model_or_local_blacklist <- glm(formula_or_local_blacklist, data = data, family = binomial)
>
> # Counterfactual predictions
> EY_a1_or_local_blacklist <- mean(predict(model_or_local_blacklist, newdata = data_all_enrolled, type =
> EY_a0_or_local_blacklist <- mean(predict(model_or_local_blacklist, newdata = data_no_enrolled, type =
> ace_or_local_blacklist <- EY_a1_or_local_blacklist - EY_a0_or_local_blacklist
>
> print(paste("Local Discovery OR ACE:", round(ace_or_local_blacklist, 4)))
```

```
[1] "Local Discovery OR ACE: 0.1869"
```

34 local algorithm nontransformed ipw

```
> if (length(cov_local_a_blacklist) == 0) {
+   formula_ps_local_blacklist <- as.formula("A ~ 1")
+ } else {
+   formula_ps_local_blacklist <- as.formula(paste("A ~", paste(cov_local_a_blacklist, collapse = " + "
+ }
>
> model_ps_local_blacklist <- glm(formula_ps_local_blacklist, data = data, family = binomial)
> data$ps_local_blacklist <- predict(model_ps_local_blacklist, type = "response")
> data$ps_local_clamped_blacklist <- pmax(pmin(data$ps_local_blacklist, 0.9999), 0.0001)
>
> # Weights
> data$weights_local_blacklist <- ifelse(data$A == 1, 1 / data$ps_local_clamped_blacklist, 1 / (1 - data
>
> a0_group <- data[data$A == 0, ]
> a1_group <- data[data$A == 1, ]
>
>
> EY_a0_ipw_local_blacklist <- sum(as.numeric(as.character(a0_group$Y)) * a0_group$weights_local_blackli
> EY_a1_ipw_local_blacklist <- sum(as.numeric(as.character(a1_group$Y)) * a1_group$weights_local_blackli
>
> ace_ipw_local_blacklist <- EY_a1_ipw_local_blacklist - EY_a0_ipw_local_blacklist
> print(paste("Local Discovery IPW ACE:", round(ace_ipw_local_blacklist, 4)))
```

```
[1] "Local Discovery IPW ACE: 0.1869"
```

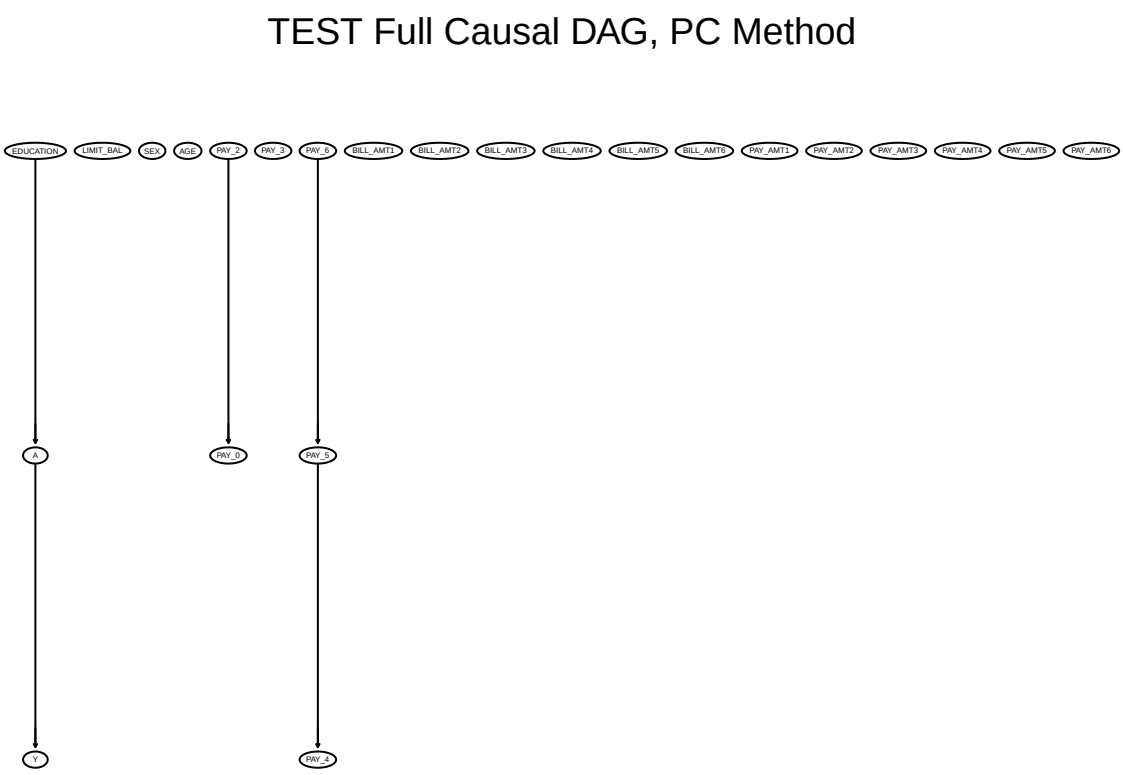
```
> orig_ace_local_var_blacklist <- c(ace_or_local_blacklist, ace_ipw_local_blacklist)
```

35 8 pc with only most recent month bnlearn, with blacklist

```
> #convert data to numerical for gaussian tests
> pc_data <- data %>%
+   mutate(across(where(is.integer), as.numeric)) %>% #convert all integers to numeric, to run
+   select(Y, A, all_of(cov_all))
>
```

```
> # calculate correlation matrix and sample size; use to test for conditional independence amongst pairs
> # pc_corr_size <- list(C = cor(pc_data), n = nrow(pc_data))
>
>
> pc_fit_bnlearn_blacklist <- pc.stable(pc_data,
+                     blacklist = master_blacklist,
+                       # test = "mix", #not needed, will automatically detect which test based on vari
+                     alpha = 0.01)
>
>
> # Get adjacency matrix from PC fit
> # pc_fit_adj_matrix <- amat(pc_fit)
> # print(pc_fit_adj_matrix)
>
> pc_fit_bnlearn_cextend_blacklist <- cextend(pc_fit_bnlearn_blacklist) #directs edges as much as possi
>
>
> pc_fit_bnlearn_maxsx_blacklist <- pc.stable(pc_data,
+                     blacklist = master_blacklist,
+                       # test = "mix", #not needed, will automatically detect which test based on vari
+                     alpha = 0.01,
+                     max.sx=2) #limits size of conditioning set to 2, which should help with the power
> pc_fit_bnlearn_maxsx_cextend_blacklist <- cextend(pc_fit_bnlearn_maxsx_blacklist) #directs edges as m
```

```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_blacklist, shape = "ellipse",
+      main = "TEST Full Causal DAG, PC Method")
```

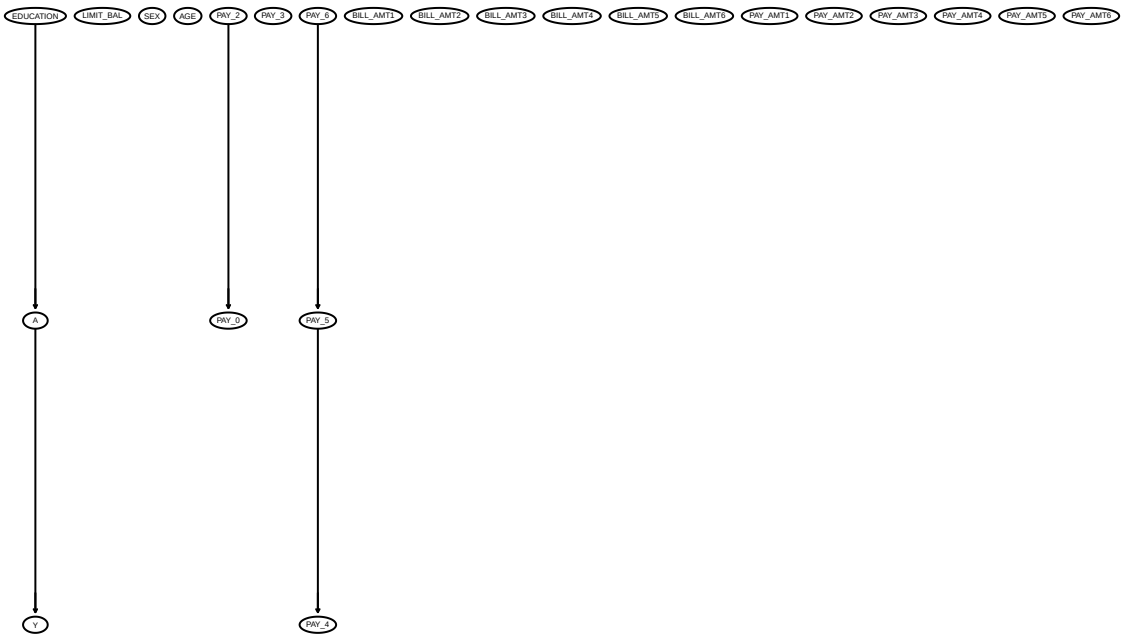



```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_blacklist_test.png", width = 1600
```

```
> graphviz.plot(pc_fit_bnlearn_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

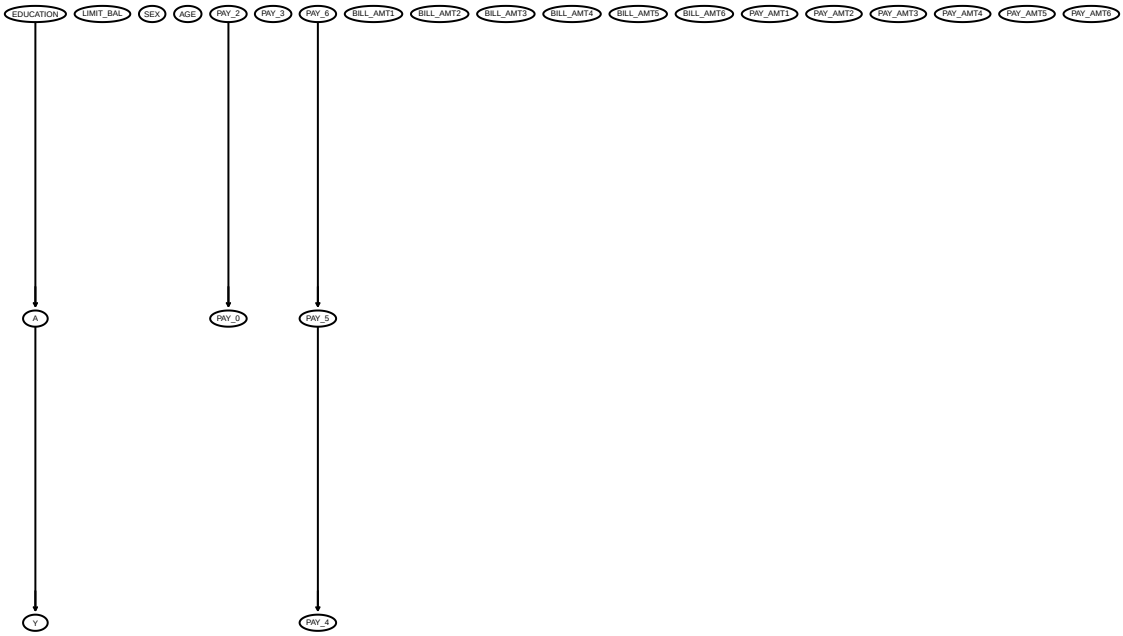
```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_cextend_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_cextend_blacklist_test.png", width
> graphviz.plot(pc_fit_bnlearn_cextend_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

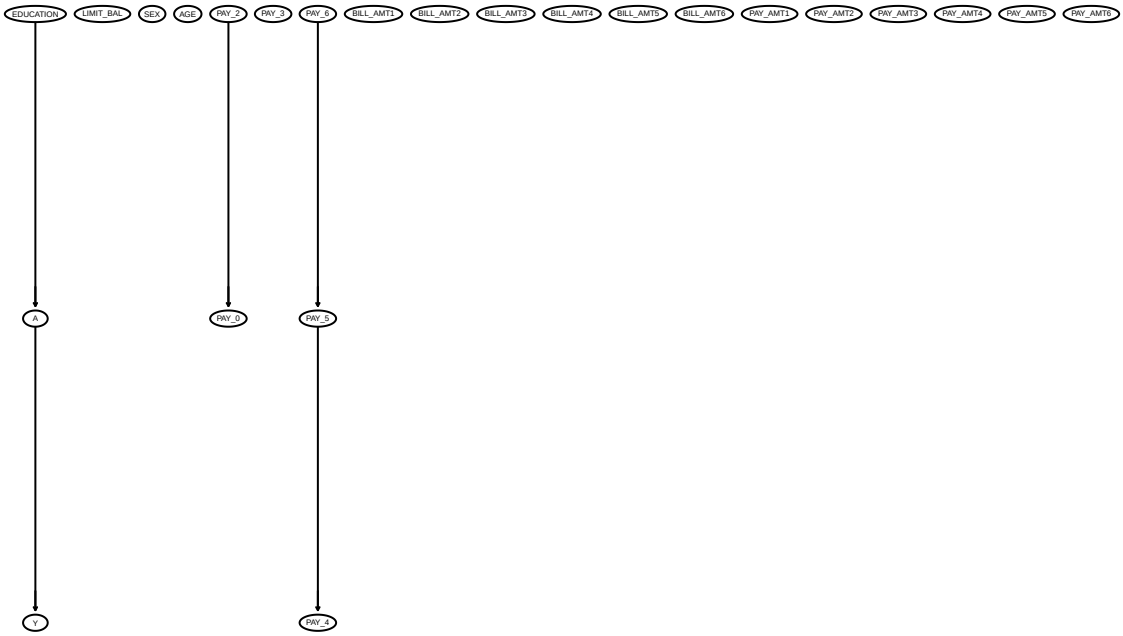
```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_maxsx_blacklist, shape = "ellipse",
+       main = "TESTFull Causal DAG, PC Method")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_maxsx_blacklist_test.png", width =
> graphviz.plot(pc_fit_bnlearn_maxsx_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

```
> # Plot DAG
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
```




```
> # 2. Export a high-quality version
> png("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/causal_dag_pc_method_bnlearn_cextend_maxsx_blacklist_test.png"
> graphviz.plot(pc_fit_bnlearn_maxsx_cextend_blacklist, shape = "ellipse",
+       main = "TEST Full Causal DAG, PC Method")
> # 3. Close the device to save the file
> dev.off()
```

```
pdf
  2
```

This doesn't work very well, no point in blacklisting...

36 9 end Note: compare the ACEs for all of the runs at the end!! maybe do a big table to discuss.

```
> # Define a helper function to label and combine
> combine_metrics <- function(df, selection_name, data_type) {
+   df %>%
+     mutate(
+       Selection = selection_name,
+       Rel_Bias_Pct = ((Bootstrap_ACE - Original_ACE) / Original_ACE) * 100
+       # Bias_SE_Ratio = abs(Bootstrap_ACE - Original_ACE) / Standard_Error
+     ) %>%
+     mutate(across(where(is.numeric), ~round(.x, 4))) %>%
+     mutate(Rel_Bias_Pct = round(Rel_Bias_Pct, 2)) %>%
+     select(Selection, Method, Original_ACE, Bootstrap_ACE, everything())
+ }
>
> # Create a master table
```

```
> all_metrics_master <- bind_rows(
+   combine_metrics(metrics_all_var, "All Variables"),
+   combine_metrics(metrics_lasso, "Lasso"),
+   combine_metrics(metrics_pc_var, "PC, Limited Covariates",),
+   combine_metrics(metrics_rank_pc_var, "Rank PC"),
+   combine_metrics(metrics_local_var, "IAMB"),
+   # combine_metrics(metrics_pc_var_full, "PC, All Covariates")
+   # combine_metrics(metrics_local_var_blacklist, "IAMB with Blacklist")
+ )
>
> print(all_metrics_master)
```

```
               Selection     Method Original_ACE Bootstrap_ACE Standard_Error
1          All Variables Regression       0.2064        0.2147         0.0439
2          All Variables        IPW       0.2256        0.2363         0.0632
3                  Lasso Regression       0.2033        0.2045         0.0401
4                  Lasso        IPW       0.1783        0.1953         0.0428
5  PC, Limited Covariates Regression       0.2074        0.2083         0.0400
6  PC, Limited Covariates        IPW       0.2064        0.2003         0.0462
7                 Rank PC Regression       0.2074        0.2029         0.0412
8                 Rank PC        IPW       0.1840        0.1858         0.0457
9                    IAMB Regression       0.2074        0.2072         0.0410
10                   IAMB        IPW       0.1791        0.1926         0.0479
   Bootstrap_Bias Rel_Bias_Pct
1          0.0083         4.03
2          0.0107         4.73
3          0.0012         0.58
4          0.0170         9.55
5          0.0009         0.42
6         -0.0061        -2.97
7         -0.0046        -2.20
8          0.0019         1.02
9         -0.0002        -0.09
10         0.0135         7.51
```

```
> # 1. Prepare the data (Rounding and Renaming)
> table_data <- all_metrics_master %>%
+   mutate(
+     # Match the decimal logic from your gt code
+     Original_ACE = round(Original_ACE, 4),
+     Bootstrap_ACE = round(Bootstrap_ACE, 4),
+     Standard_Error = round(Standard_Error, 4),
+     Bootstrap_Bias = round(Bootstrap_Bias, 4),
+     Rel_Bias_Pct = paste0(round(Rel_Bias_Pct, 1), "%") # Format as string for ggplot
+     # Bias_SE_Ratio = round(Bias_SE_Ratio, 3)
+   ) %>%
+   # Rename columns to match your desired header
+   rename(
+     `Estimation Method` = Method,
+     `Original ACE` = Original_ACE,
+     `Bootstrap Mean ACE` = Bootstrap_ACE,
+     `Bootstrap Standard Error (SE)` = Standard_Error,
+     `Bootstrap Bias` = Bootstrap_Bias,
+     `Relative Bootstrap Bias (%)` = Rel_Bias_Pct,
```

```
+     # `Bootstrap Bias-to-SE Ratio` = Bias_SE_Ratio
+   )
>
> # 2. Create the table graphic
> # We use 'ttheme_minimal' or 'ttheme_clean' for a thesis look
> tab_plot <- ggtexttable(table_data,
+                         rows = NULL,
+                         theme = ttheme("classic", base_size = 10)) %>%
+             tab_add_title(text = "ACE Estimation Comparison Across Variable Selection Methods",
+                           face = "bold", padding = unit(1.5, "line"))
>
> # 3. Save as PNG using ggsave (No Chrome needed!)
> ggsave(filename = "OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/ACE_Comparison_Table.png",
+        plot = tab_plot,
+        width = 12, height = 8, dpi = 300, bg = "white")
>
> # View it in RStudio
> print(tab_plot)
```

ison Across Variable Selection Methods

ion Method	Original ACE	Bootstrap Mean ACE	Bootstrap Standard Error (SE)	Bootstrap Bia
gression	0.2064	0.2147	0.0439	0.0083
IPW	0.2256	0.2363	0.0632	0.0107
gression	0.2033	0.2045	0.0401	0.0012
IPW	0.1783	0.1953	0.0428	0.0170
gression	0.2074	0.2083	0.0400	0.0009
IPW	0.2064	0.2003	0.0462	−0.0061
gression	0.2074	0.2029	0.0412	−0.0046
IPW	0.1840	0.1858	0.0457	0.0019
gression	0.2074	0.2072	0.0410	−0.0002
IPW	0.1791	0.1926	0.0479	0.0135

```
> # # 1. Create the professional gt table object
> # final_table_img <- all_metrics_master %>%
> #   gt(groupname_col = "Selection") %>% # Group by Selection method
>
> #   # Add title and subtitle for your thesis
> #   tab_header(
> #     title = md("ACE Estimation Comparison Across Variable Selection Methods"),
> #   ) %>%
>
> #   # Format column labels clearly
> #   cols_label(
> #     Method = md("Estimation Method"),
> #     Original_ACE = md("Original ACE"),
> #     Bootstrap_ACE = md("Bootstrap Mean ACE"),
> #     Standard_Error = md("Bootstrap Standard Error (SE)"),
```

```
> #     Rel_Bias_Pct = md("Relative Bootstrap Bias (%)"),
> #     Bias_SE_Ratio = md("Bootstrap Bias-to-SE Ratio"),
> #     Bootstrap_Bias = md("Bootstrap Bias")
> #   ) %>%
>
> #   # Add formatting for scientific precision and percentages
> #   fmt_number(columns = c(Original_ACE, Bootstrap_ACE, Standard_Error), decimals = 4) %>%
> #   fmt_percent(columns = Rel_Bias_Pct, decimals = 1, scale_values=FALSE) %>%
> #   fmt_number(columns = Bias_SE_Ratio, decimals = 3) %>%
>
> #   # Choose a clean theme (like minimal or pff)
> #   gt_theme_pff()
>
> # # 2. Save the table as an image (PNG)
> # # Ensure webshot2 is installed for this step
> # gtsave(final_table_img, filename = "OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/ACE_Comparison_Table.png")
>
> # # Print the table to the Viewer to see it immediately
> # final_table_img
```

```
> print("Formula for ordinary regression model for all variables:")
```

```
[1] "Formula for ordinary regression model for all variables:"
```

```
> print(formula_or_all_var)
```

```
Y ~ A + LIMIT_BAL + SEX + EDUCATION + AGE + PAY_0 + PAY_2 + PAY_3 +
    PAY_4 + PAY_5 + PAY_6 + BILL_AMT1 + BILL_AMT2 + BILL_AMT3 +
    BILL_AMT4 + BILL_AMT5 + BILL_AMT6 + PAY_AMT1 + PAY_AMT2 +
    PAY_AMT3 + PAY_AMT4 + PAY_AMT5 + PAY_AMT6
```

```
> print("Formula for IPW model for all variables:")
```

```
[1] "Formula for IPW model for all variables:"
```

```
> print(formula_ps_all_var)
```

```
A ~ LIMIT_BAL + SEX + EDUCATION + AGE + PAY_0 + PAY_2 + PAY_3 +
    PAY_4 + PAY_5 + PAY_6 + BILL_AMT1 + BILL_AMT2 + BILL_AMT3 +
    BILL_AMT4 + BILL_AMT5 + BILL_AMT6 + PAY_AMT1 + PAY_AMT2 +
    PAY_AMT3 + PAY_AMT4 + PAY_AMT5 + PAY_AMT6
```

```
> print("Formula for ordinary regression model for lasso:")
```

```
[1] "Formula for ordinary regression model for lasso:"
```

```
> print("It gets cut off so I just output the covariates instead of a full formula as well")
```

```
[1] "It gets cut off so I just output the covariates instead of a full formula as well"
```

```
> print(formula_lasso_or_text)
```

```
[1] "Y ~  A1 + LIMIT_BAL + EDUCATION2 + EDUCATION4 + AGE + PAY_0-1 + PAY_00 + PAY_01 + PAY_02 + PAY_03 +
```

```
> print(formula_lasso_or_text_anotherver)
```

```
 [1] "A1"         "LIMIT_BAL"  "EDUCATION2" "EDUCATION4" "AGE"
 [6] "PAY_0-1"    "PAY_00"     "PAY_01"     "PAY_02"     "PAY_03"
[11] "PAY_06"     "PAY_24"     "PAY_25"     "PAY_32"     "PAY_34"
```

```
[16] "PAY_4-1"    "PAY_42"     "PAY_43"     "PAY_54"     "PAY_62"
[21] "BILL_AMT5"  "PAY_AMT2"   "PAY_AMT3"   "PAY_AMT4"   "PAY_AMT6"
```

```
> print("Formula for IPW model for lasso variable selection:")
```

```
[1] "Formula for IPW model for lasso variable selection:"
```

```
> print("It gets cut off so I just output the covariates instead of a full formula as well")
```

```
[1] "It gets cut off so I just output the covariates instead of a full formula as well"
```

```
> print(formula_lasso_ps_text)
```

```
[1] "A ~ SEX1 + EDUCATION3 + EDUCATION5 + AGE + PAY_04 + PAY_AMT6"
```

```
> print(formula_lasso_ps_text_anotherver)
```

```
[1] "SEX1"       "EDUCATION3" "EDUCATION5" "AGE"        "PAY_04"
[6] "PAY_AMT6"
```

```
> print("Formula for ordinary regression model for PC, limiting to most recent month:")
```

```
[1] "Formula for ordinary regression model for PC, limiting to most recent month:"
```

```
> print(formula_or_pc)
```

```
Y ~ A + PAY_0
```

```
> print("Formula for IPW for PC, limiting to most recent month:")
```

```
[1] "Formula for IPW for PC, limiting to most recent month:"
```

```
> print(formula_ps_pc_var)
```

```
A ~ PAY_0 + EDUCATION
```

```
> print("Formula for ordinary regression model for Rank PC:")
```

```
[1] "Formula for ordinary regression model for Rank PC:"
```

```
> print(formula_or_rank_pc)
```

```
Y ~ A + PAY_0
```

```
> print("Formula for IPW for Rank PC:")
```

```
[1] "Formula for IPW for Rank PC:"
```

```
> print(formula_ps_rank_pc_var)
```

```
A ~ AGE + LIMIT_BAL
```

```
> print("Formula for ordinary regression model for local discovery (INTER_IAMB)")
```

```
[1] "Formula for ordinary regression model for local discovery (INTER_IAMB)"
```

```
> print(formula_or_local)
```

```
Y ~ A + PAY_0
```

```
> print("formula for IPW for local discovery (INTER_IAMB)")
```

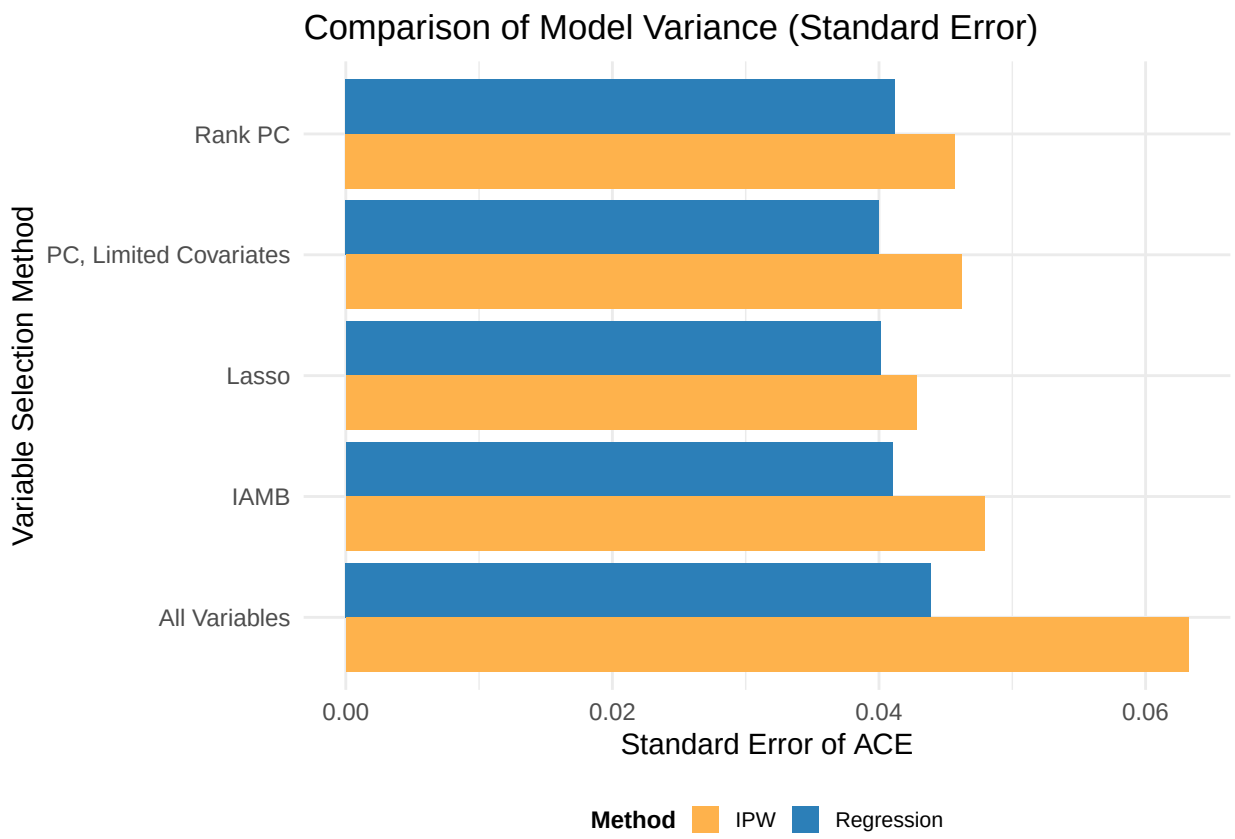
```
[1] "formula for IPW for local discovery (INTER_IAMB)"
```

```
> print(formula_ps_local)
```

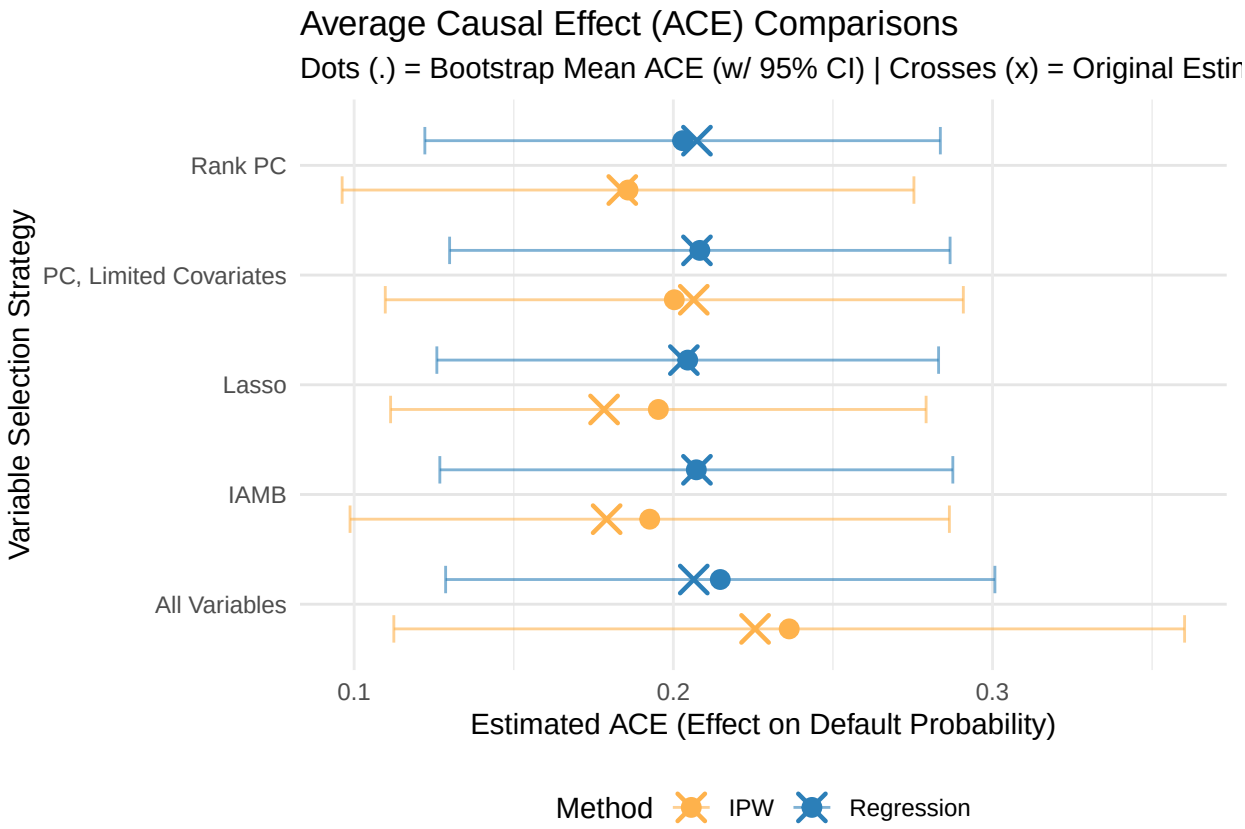
```
A ~ AGE + EDUCATION
```

```
> # print("Formula for ordinary regression model for PC with ALL covariates")
> # print(formula_or_pc_full)
> # print("formula for IPW for PC with ALL covariates")
> # print(formula_ps_pc_var_full)
```

```
> # Calculate 95% Confidence Intervals using the Normal Approximation: ACE +/- 1.96 * SE
> all_metrics_master <- all_metrics_master %>%
+  mutate(
+     CI_lower = Bootstrap_ACE - (1.96 * Standard_Error),
+     CI_upper = Bootstrap_ACE + (1.96 * Standard_Error)
+   )
> ace_se_graph <- ggplot(all_metrics_master, aes(x = Selection, y = Standard_Error, fill = Method)) +
+   geom_bar(stat = "identity", position = "dodge") +
+   scale_fill_manual(values = c("Regression" = "#2c7fb8", "IPW" = "#feb24c")) +
+   coord_flip() +
+   theme_minimal() +
+   labs(title = "Comparison of Model Variance (Standard Error)",
+        y = "Standard Error of ACE", x = "Variable Selection Method")+
+   theme(
+     legend.position = "bottom",
+     legend.text = element_text(size = 8),     # Slightly smaller text to save space
+     legend.title = element_text(size = 9, face = "bold"),
+     legend.key.size = unit(0.4, "cm")         # Smaller keys to fit more on screen
+   )
> ggsave("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/ace_standard_errors.png", plot = ace_se_graph, width = 8, h
> print(ace_se_graph)
```




```
> ace_ci_graph <- ggplot(all_metrics_master, aes(x = Selection, y = Bootstrap_ACE, color = Method, grou
+   geom_errorbar(aes(ymin = CI_lower, ymax = CI_upper),
+                 position = position_dodge(width = 0.9),
+                 width = 0.5, alpha = 0.6) +
+   geom_point(aes(y = Bootstrap_ACE),
+              position = position_dodge(width = 0.9),
+              size = 3) +
+   geom_point(aes(y = Original_ACE),
+              shape = 4,
+              position = position_dodge(width = 0.9),
+              size = 4, stroke = 1.2) +
+   coord_flip() +
+   theme_minimal() +
+   scale_color_manual(values = c("Regression" = "#2c7fb8", "IPW" = "#feb24c")) +
+   labs(title = "Average Causal Effect (ACE) Comparisons",
+        subtitle = "Dots (•) = Bootstrap Mean ACE (w/ 95% CI) | Crosses (x) = Original Estimate of ACE
+        y = "Estimated ACE (Effect on Default Probability)",
+        x = "Variable Selection Strategy",
+        color = "Method",
+        shape = "Data Format") +
+   theme(
+     legend.position = "bottom",
+     panel.grid.major.y = element_line(color = "grey90") # Makes the rows easier to follow
+   )
>
> ggsave("OUTPUT_IMAGES_FROM_ANALYSIS_V6_01/ace_confidence_intervals.png", plot = ace_ci_graph, width =
> print(ace_ci_graph)
```


Average Causal Effect (ACE) Comparisons
Dots (.) = Bootstrap Mean ACE (w/ 95% CI) | Crosses (x) = Original Estir
Variable Selection Strategy
Rank PC
PC, Limited Covariates
Lasso
IAMB
All Variables
0.1
0.2
0.3
Estimated ACE (Effect on Default Probability)
Method
IPW
Regression